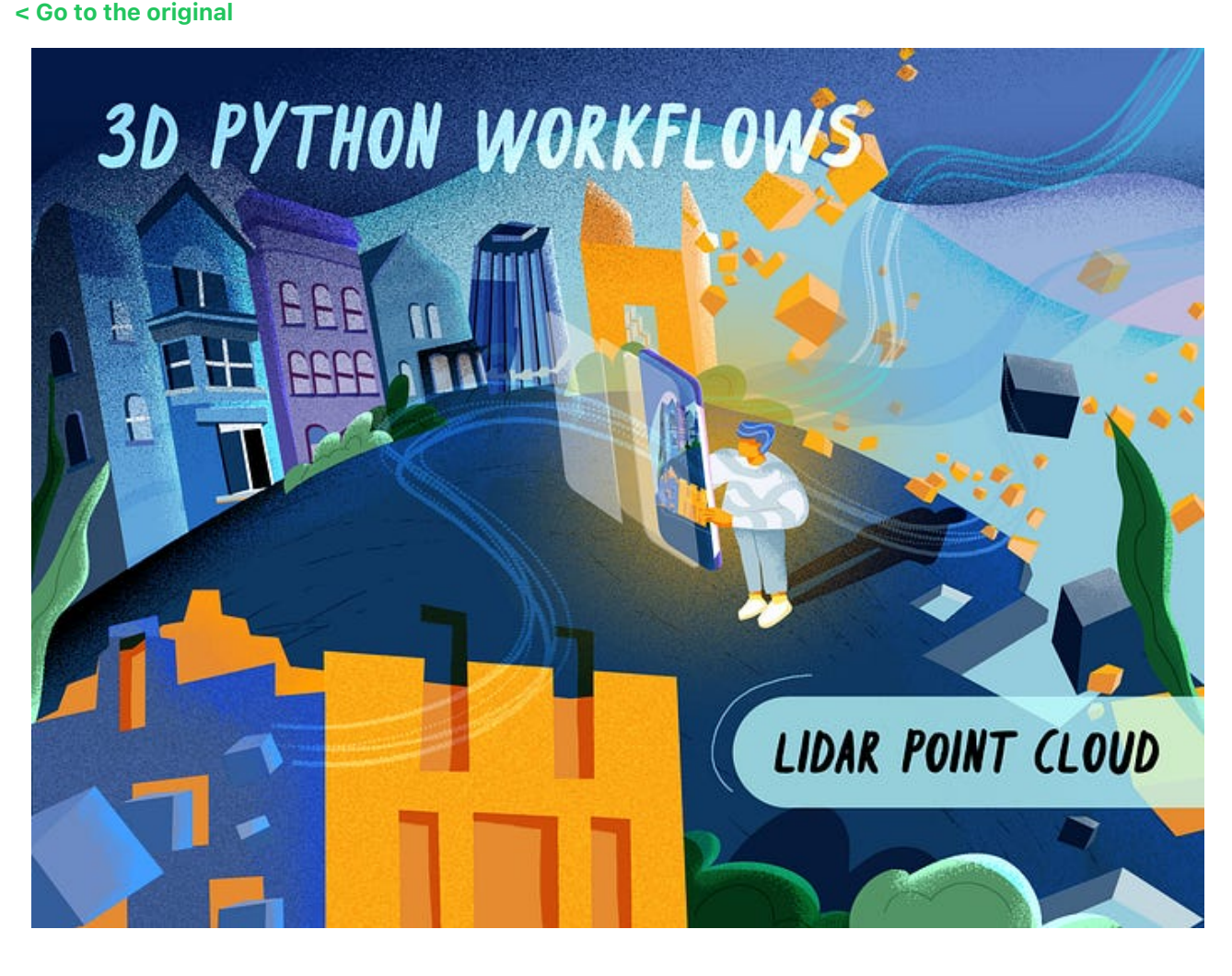
< Go to the original


3D PYTHON WORKFLOWS
LIDAR POINT CLOUD

# 3D Python Workflows for LiDAR City Models: A Step-by-Step Guide

The Ultimate Guide to unlocking a streamlined workflow for 3D City Modelling Applications. The tutorial covers Python Automation combining...

**Florent Poux, Ph.D.**

Follow

Towards Data Science · a11y-light · ~38 min read · April 4, 2023 (Updated: May 15, 2023) · Free: No

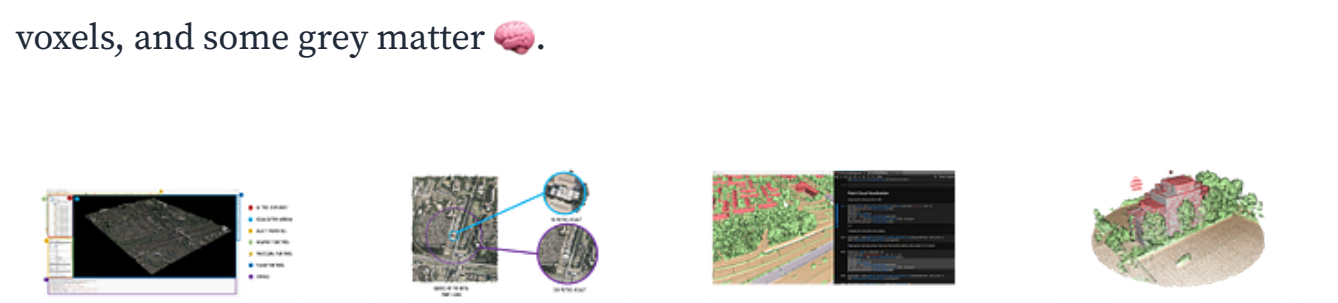
### 3D Python

The Ultimate Guide to unlocking a streamlined workflow for 3D City Modelling Applications. The tutorial covers Python Automation combining 3D Point Clouds, Meshes, and Voxels for advanced analysis.

Did you stumble upon the term Smart City before? or Smart Something? Well, we touch on the subject! Think of a Smart City as a baker on steroids 🥐: it knows what you need before you even ask for it and will provide you with the most straightforward advice to make a delicious choice. No, this Smart City Metaphor is not all I have for you today. Indeed, to get to this level of "Smart", we first have to get to the base layer: 3D City Models.

If you ever wanted to create stunning 3D City Models but found the workflow daunting and complex, this is where I come in! This article explores how Python and open-source software can define a powerful 3D workflow to kickstart your 3D City Modelling journey. Say (almost) goodbye to tedious manual processes and hello to efficient, dynamic, and eye-catching creations!

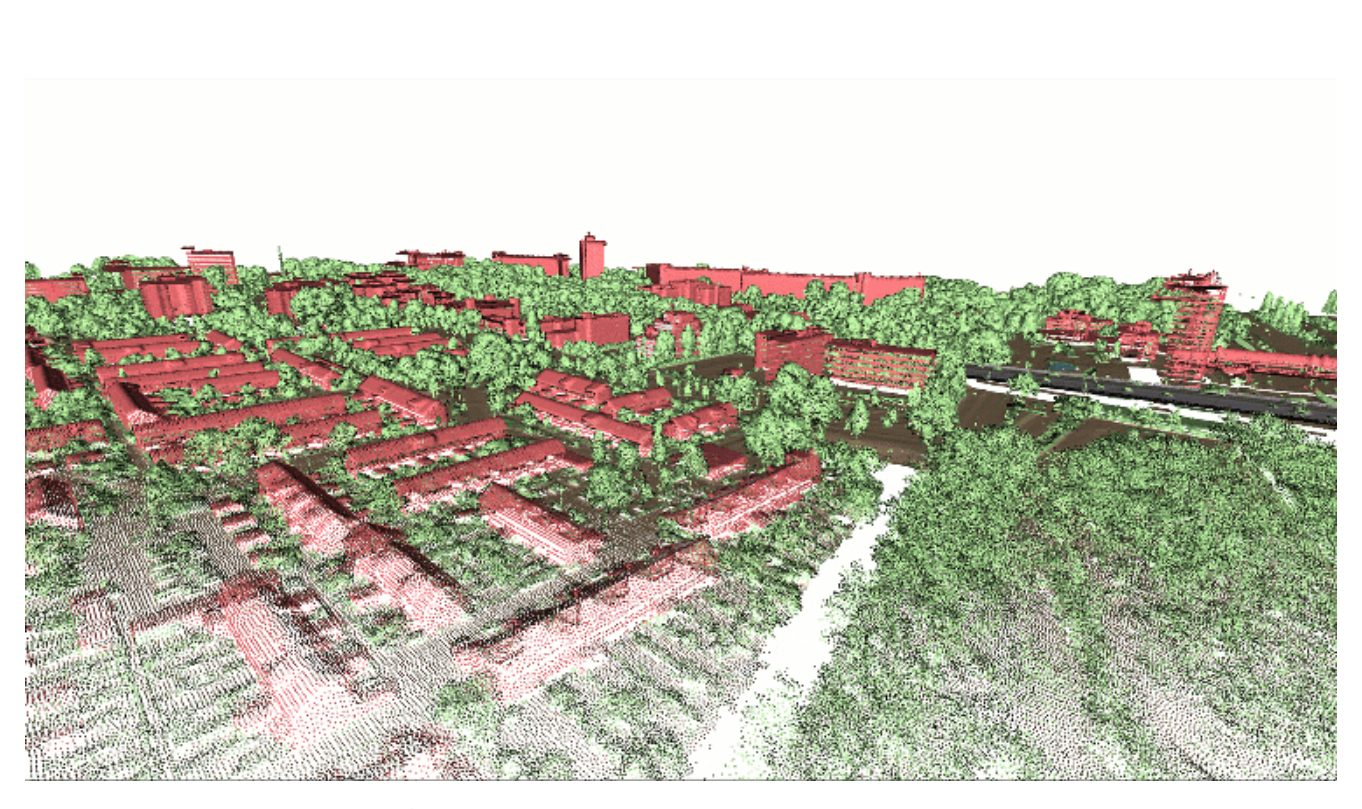
We dive into a four-step strategy that describes Environment setup, 3D Data Curation & Preparation, and 3D Geometry Processing to extract critical insights such as the built coverage of your neighborhood using point cloud data, meshes, voxels, and some grey matter 🧠.


1. ENVIRONMENT SET-UP
2. 3D DATA PREPARATION
3. 3D PYTHON AUTOMATION
4. 3D PYTHON CHALLENGES

3D Python Workflows for LiDAR City Models: A Step-by-Step Guide. © Author

If you are ready and pumped, it is time to get 3D Coding!

🎵 **Note to Readers**: *This hands-on guide is part of a* ***UTWENTE*** *joint work with my dear colleagues* ***Dr. Sander Oude Elberink***, ***Dr. Mila Koeva***, ***Dr. Ville Lehtola***, ***Dr. Pirouz Nourian***, ***Dr. Paulo Raposo***, *and* ***Prof G. Vosselman***. *We acknowledge the financial contribution from the digital twins @ITC -project granted by the ITC faculty of the University of Twente.*

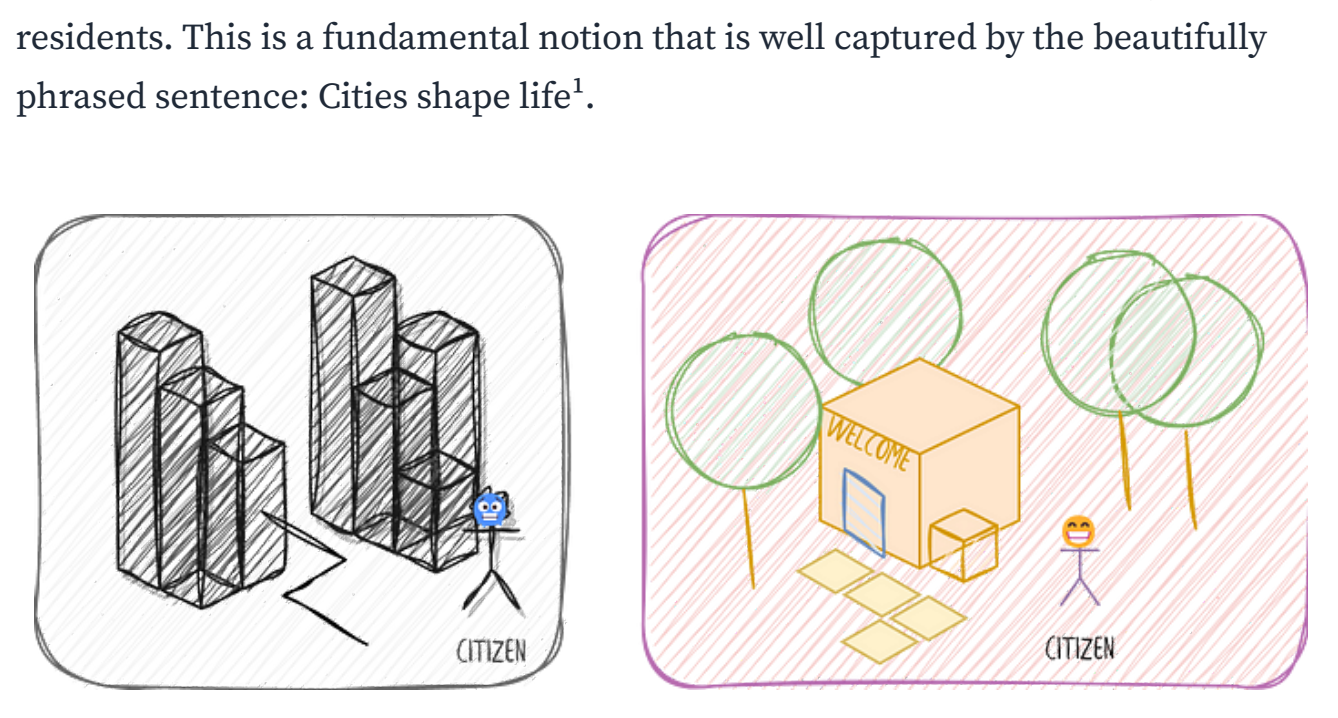


An extract of the 3D Python dataset we will handle in this guide. © Author

## Introduction

Before rushing onto the fun bits, let me tell you a small story to give a bit of depth to what we will achieve. This starts with a fundamental question: What is 3D City Modeling, and why is it helpful?

In an urbanized world, 3D city modeling is essential for efficiently managing our daily lives. By accurately representing our cities in three dimensions, we can analyze and visualize complex urban environments, understand the impact of proposed changes, and make informed decisions to improve the quality of life for residents. This is a fundamental notion that is well captured by the beautifully phrased sentence: Cities shape life[1].


CITIZEN
WELCOME
CITIZEN

Toward Smart Cities to improve our lives. © Author

Indeed, Cities are places where people live, form communities, and establish their own identities. They are spaces, such as the inner city and the suburb, that offer a way to configure and shape the material world and natural environment.

Imagine if you had a superpowered ability to model transportation networks in your city, predict traffic patterns and identify areas of congestion. How would it change the way you live your city? And that is a super tiny example taken as a city "user". But at the root, 3D city modeling provides valuable insights for urban planners, architects, and policymakers to optimize city infrastructure, reduce traffic, and enhance public safety.

By creating a digital replica of our cities, we can thus better plan for the future and create sustainable, livable communities for generations to come.[2,3]

> [1] Chen, X., Orum, A. M., & Paulsen, K. E. (2018). *Introduction to Cities: How place and space shape human experience*. John Wiley & Sons.

> [2] Lehtola, V. V., Koeva, M., Elberink, S. O., Raposo, P., Virtanen, J. P., Vahdatikhaki, F., & Borsci, S. (2022). *Digital twin of a city: Review of technology serving city needs.* International Journal of Applied Earth Observation and Geoinformation, 102915.

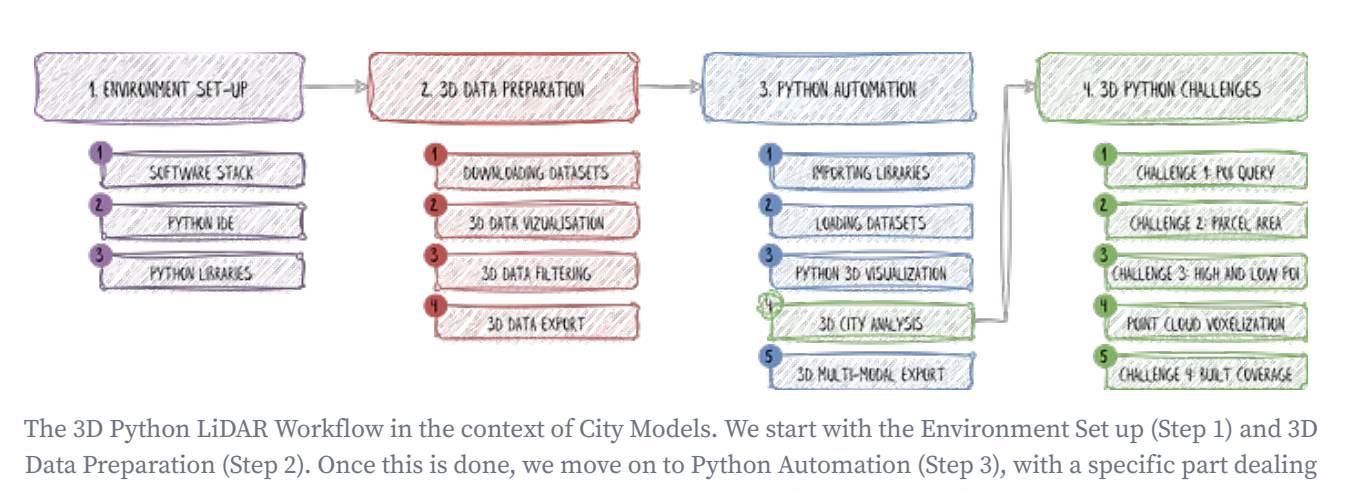
> [3] Nourian, P., Ohori, K. A., & Martinez-Ortiz, C. (2018). Essential means for urban computing: Specification of web-based computing platforms for urban planning, a Hitchhiker's guide. Urban Planning, 3(1), 47–57

## 3D City Modelling: The Workflow

Time to get half-serious and define a coherent 3D Workflow that we could use as an inspiration for different 3D City Modeling Operations. We aim at something that is (1) Easy to set up and run, (2) Provides great flexibility to various scenarios, and (3) powerful enough to encompass the complexity of multi-modal applications.

🦚 **Note**: *No, multi-modal is not a swearword: it just touches on the point that when dealing with 3D City Models, we encounter various geospatial data modalities to be considered. In this tutorial, we will focus on a beautiful niche: 3D Geospatial Data in the form of* ***Point Clouds***, ***Voxels***, *and* ***3D Meshes***.

If we decompose the workflow definition on a high-level view, we follow four main steps, as shown below.


1. ENVIRONMENT SET-UP
SOFTWARE STACK
PYTHON IDE
PYTHON LIBRARIES
2. 3D DATA PREPARATION
DOWNLOADING DATASETS
3D DATA VIZUALISATION
3D DATA FILTERING
3D DATA EXPORT
3. PYTHON AUTOMATION
IMPORTING LIBRARIES
LOADING DATASETS
PYTHON 3D VISUALIZATION
3D CITY ANALYSIS
3D MULTI-MODAL EXPORT
4. 3D PYTHON CHALLENGES
CHALLENGE 1: POI QUERY
CHALLENGE 2: PARCEL AREA
CHALLENGE 3: HIGH AND LOW POI
POINT CLOUD VOXELIZATION
CHALLENGE 4: BUILT COVERAGE

The 3D Python LiDAR Workflow in the context of City Models. We start with the Environment Set up (Step 1) and 3D Data Preparation (Step 2). Once this is done, we move on to Python Automation (Step 3), with a specific part dealing with 3D Python Challenges (Step 4), such as Parcel Surface or Point Of Interest Queries.

Excited? Looking closely at the pipeline, you can see that we start from scratch. This permits the adaptation of the proposed structure to various scenarios, which would necessitate various environments, datasets, or automation bits.

Let us now go a bit deeper. Let us imagine we own a house in the Netherlands (It can be close to the University of Twente), and we would like to understand the surrounding area better. That is our starting point. Now, to better grasp the relationship of our house to the surrounding, some questions arise: how dense is the built area of the neighborhood? Can the house be subject to flooding? Am I respecting the built ratio for the parcel I own?

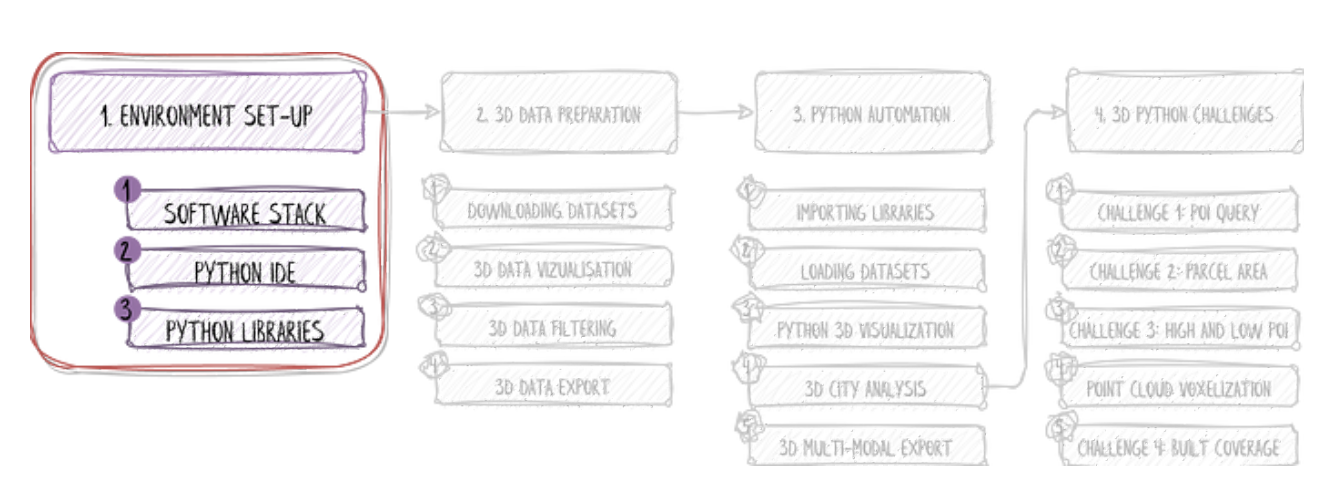
How should we go around and answer these questions? Should we look on the internet? Should we open a map? Should we call the cadastral services? Let's work around that together and simultaneously unlock a new robust set of skills in this context.

In the following, we detail the four steps that allow answering these questions: the first step covers the ideal environment setup. Secondly, we get our hands on 3D point clouds and city models as meshes of an area of interest. Then, we create a 3D Python notebook with a high focus on automation. Finally, we create a set of Python functions to answer the challenging scenarios. Okay, let us arm ourselves with coffee or tea 🍵 and dive into finding answers to these interrogations!

🦚 **Note**: *I designed the next series of actions to be easy to follow linearly without needing coding skills or available data. Nevertheless, if you are an experienced coder, I provided helpful optimization tricks on crucial tasks to ensure code performance is at its peak!*

## Step 1: 3D Environment Set-up

Before making our hands dirty with code and 3D thoughts, a good practice is to ensure we work in a proper environment. We do not cook on a dirty countertop with dull knives and outdated food (or at least we avoid 😁)! Let us follow the three sub-steps as illustrated below.

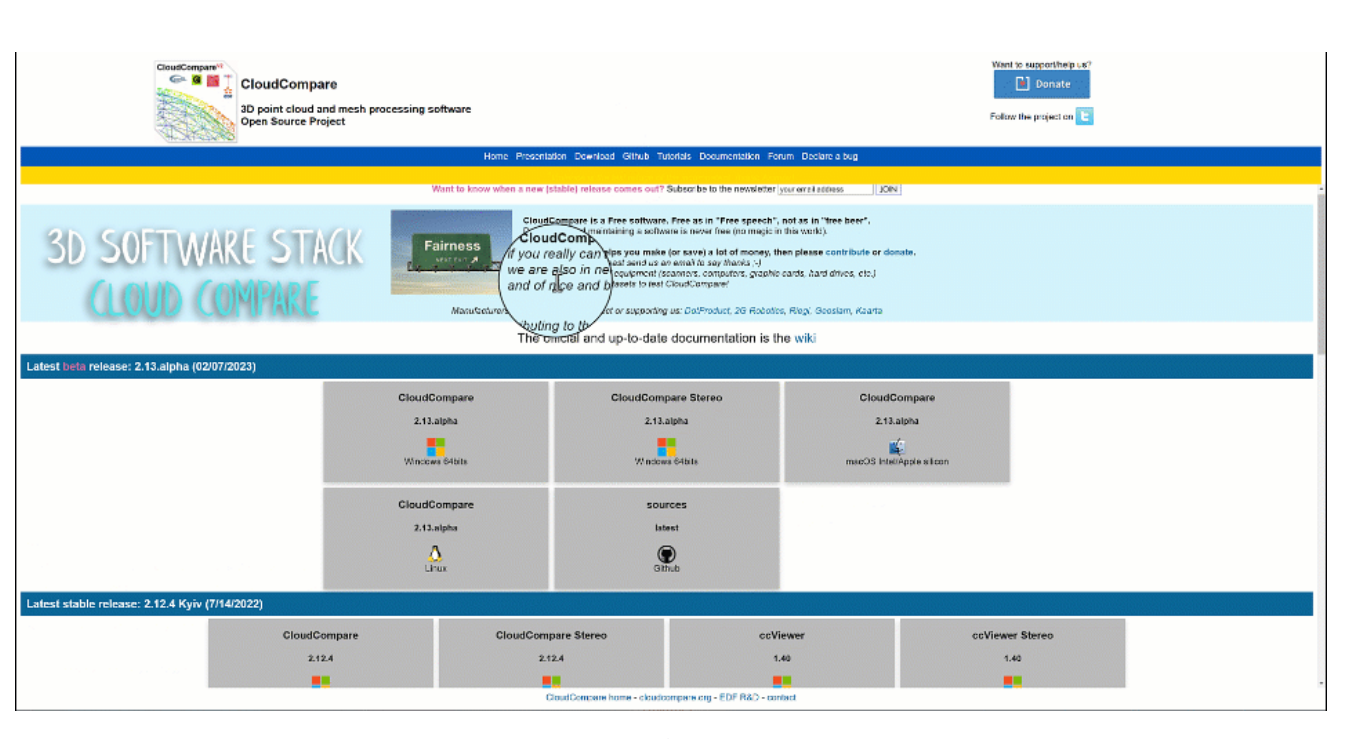

1. ENVIRONMENT SET-UP
1 SOFTWARE STACK
2 PYTHON IDE
3 PYTHON LIBRARIES
2. 3D DATA PREPARATION
DOWNLOADING DATASETS
3D DATA VIZUALISATION
3D DATA FILTERING
3D DATA EXPORT
3. PYTHON AUTOMATION
IMPORTING LIBRARIES
LOADING DATASETS
PYTHON 3D VISUALIZATION
3D CITY ANALYSIS
3D MULTI-MODAL EXPORT
4. 3D PYTHON CHALLENGES
CHALLENGE 1: POI QUERY
CHALLENGE 2: PARCEL AREA
CHALLENGE 3: HIGH AND LOW POI
POINT CLOUD VOXELIZATION
CHALLENGE 4: BUILT COVERAGE

Step 1: 3D Environment Set-up.

### 1. 1. The software stack

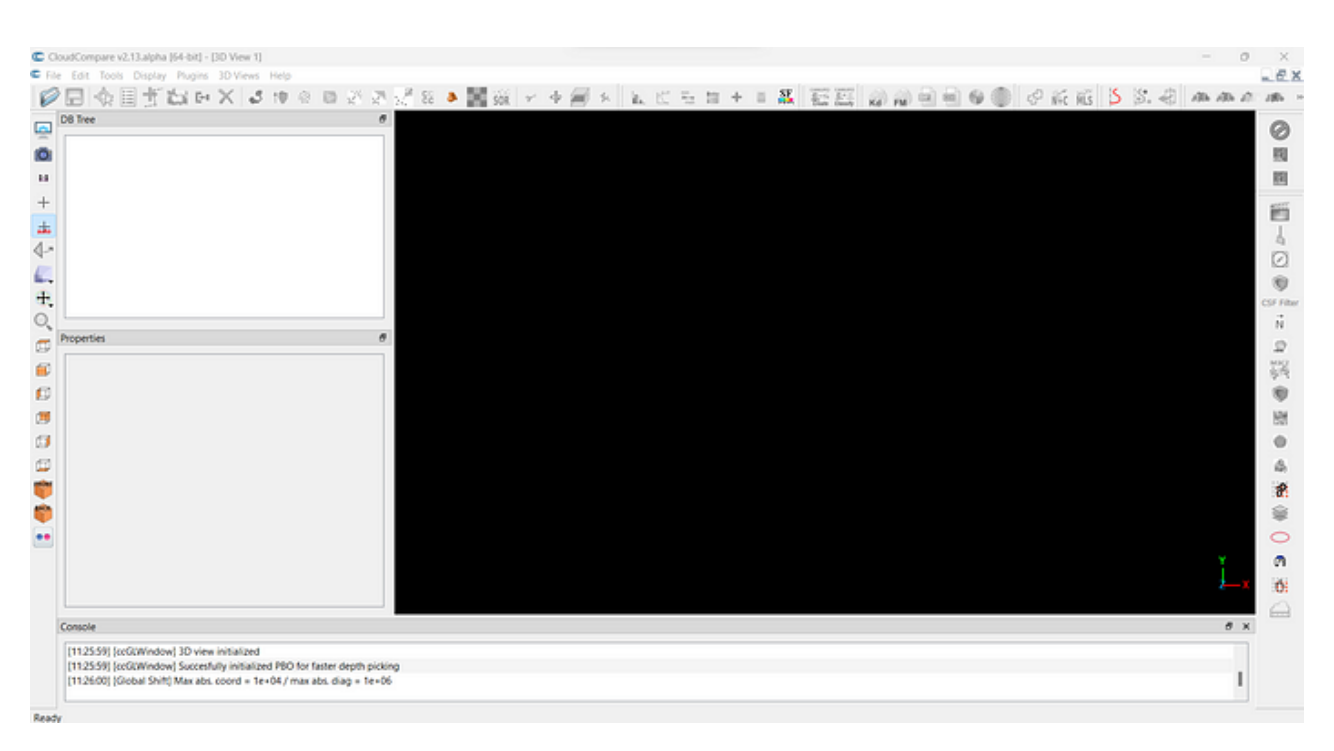
To get things going, let us first install a 3D point cloud and mesh processing software: CloudCompare. It is a marvelous tool that permits to efficiently handle the scientific analysis of point cloud data (but not only). It is an essential cog in any iterative experiment where you want to quickly get a data-driven idea about the feasibility of a theory, for example.

To get the CloudCompare software, you can go to the download section of CloudCompare.org and get the latest stable release for your OS, as illustrated below.


CloudCompare
3D point cloud and mesh processing software
Open Source Project
Donate
3D SOFTWARE STACK
CLOUD COMPARE
Latest beta release: 2.13.alpha (02/07/2023)
CloudCompare 2.13.alpha Windows 64 bits
CloudCompare Stereo 2.13.alpha Windows 64 bits
CloudCompare 2.13.alpha macOS Intel/Apple silicon
CloudCompare 2.13.alpha Linux
sources latest Github
Latest stable release: 2.12.4 Kyiv (7/14/2022)
CloudCompare 2.12.4
CloudCompare Stereo 2.12.4
ccViewer 1.40
ccViewer Stereo 1.40

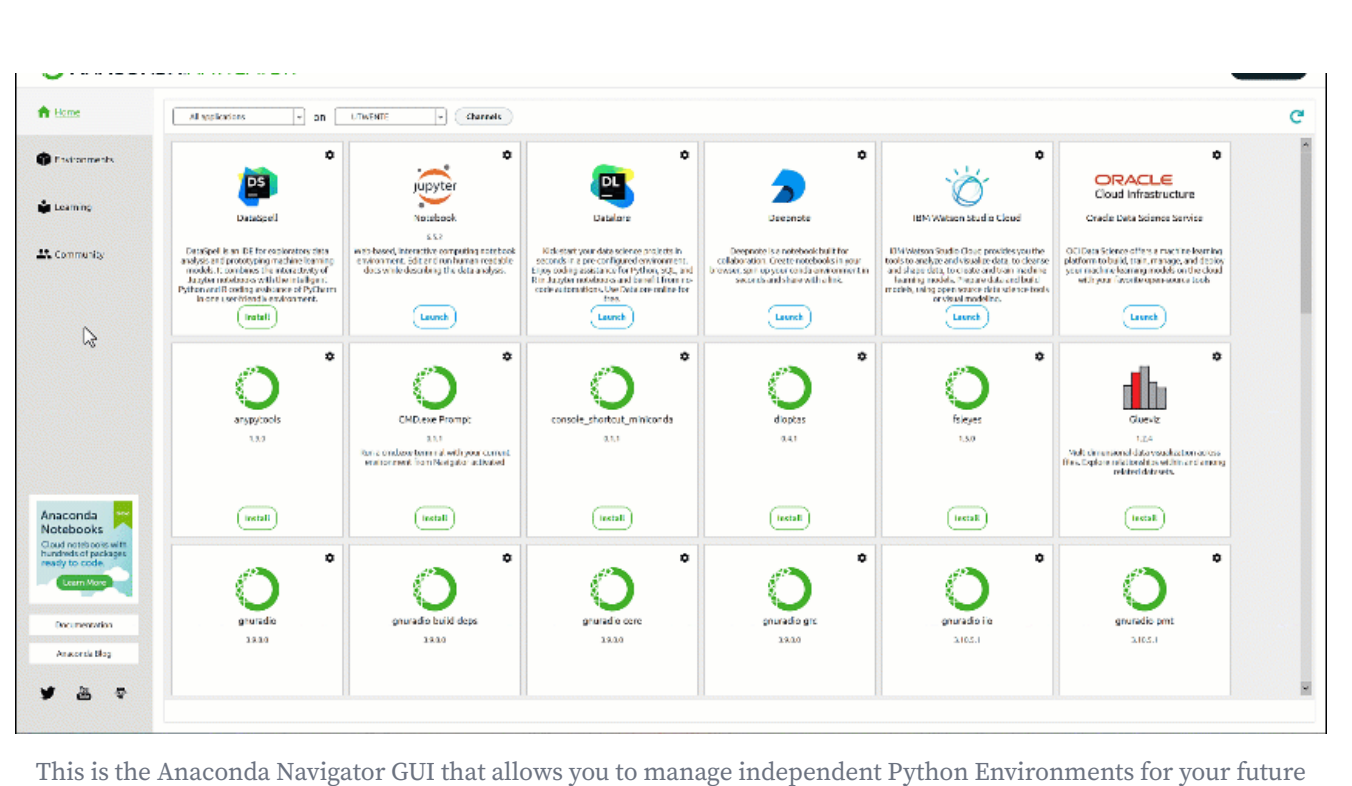
Downloading CloudCompare from https://cloudcompare.org/

After downloading the software, follow the linear installation steps to get to a working software that you can open, which should look similar to the following GUI upon launch:


DB Tree
Properties
Console

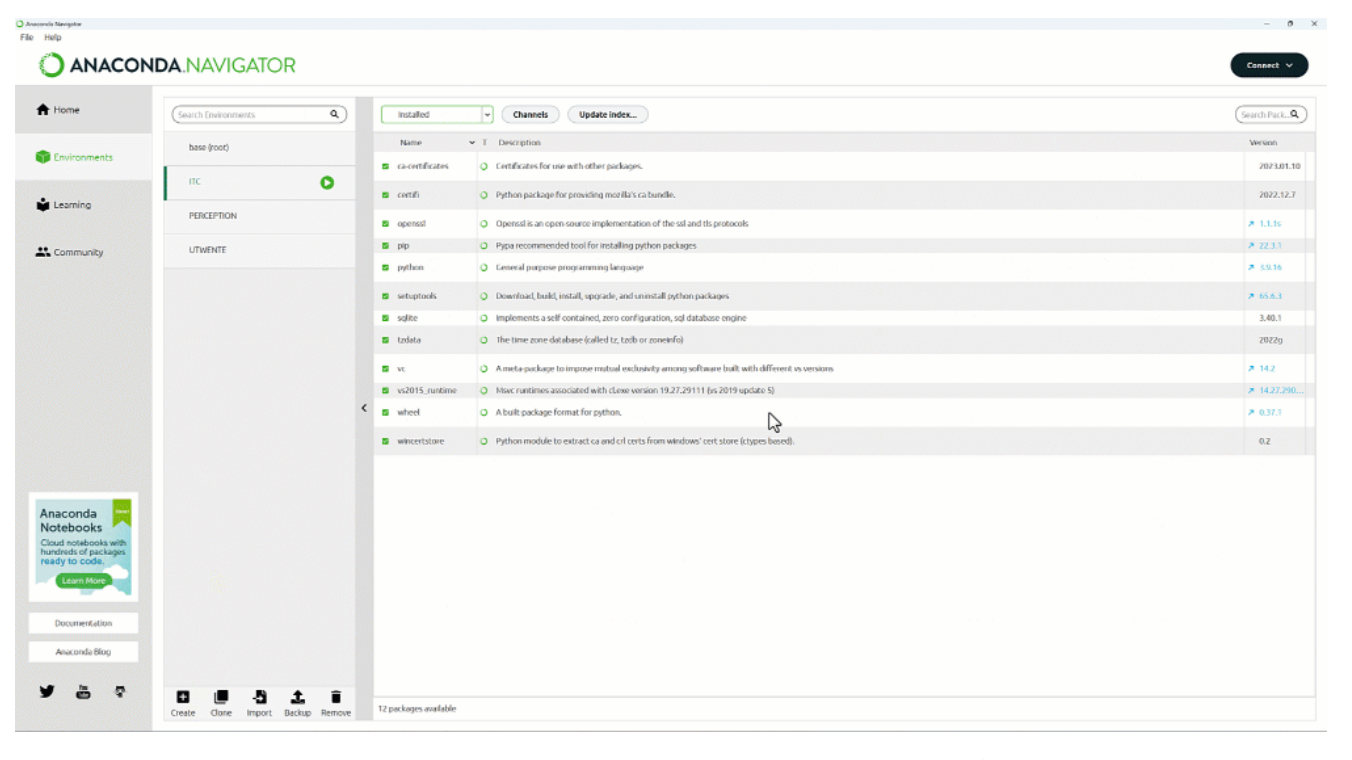
CloudCompare GUI.

Then, we want to get a Python Distribution that allows us to focus on the actual code. It is called Anaconda, and it is available on various platforms from Anaconda.com. Once you download the software that matches your OS, you can install it. A GUI is provided (Anaconda Navigator) that you can launch to get up and running quickly, as shown below..


Home
Environments
Learning
Community
Anaconda Notebooks
DataSpell
Jupyter
Datalore
IBM Watson Studio Cloud
ORACLE Cloud Infrastructure

This is the Anaconda Navigator GUI that allows you to manage independent Python Environments for your future experiments.

Using the GUI Anaconda Navigator, go to the "`Environments`" Tab on the left. We then create a brand new isolated Python environment by clicking on "`Create`" as shown below.


ANACONDA.NAVIGATOR
Home
Environments
Learning
Community
Anaconda Notebooks
Create
Clone
Import
Backup
Remove

In Anaconda Navigator, you have four tabs on the left side. In the Home Tab, you will find the IDE at your disposal in a specific environment; in the "Environments" tab, you can create, manage, and select any environment.

This will allow us to manage better some Python "libraries" we want to install (Step 1.3) while avoiding version conflicts.

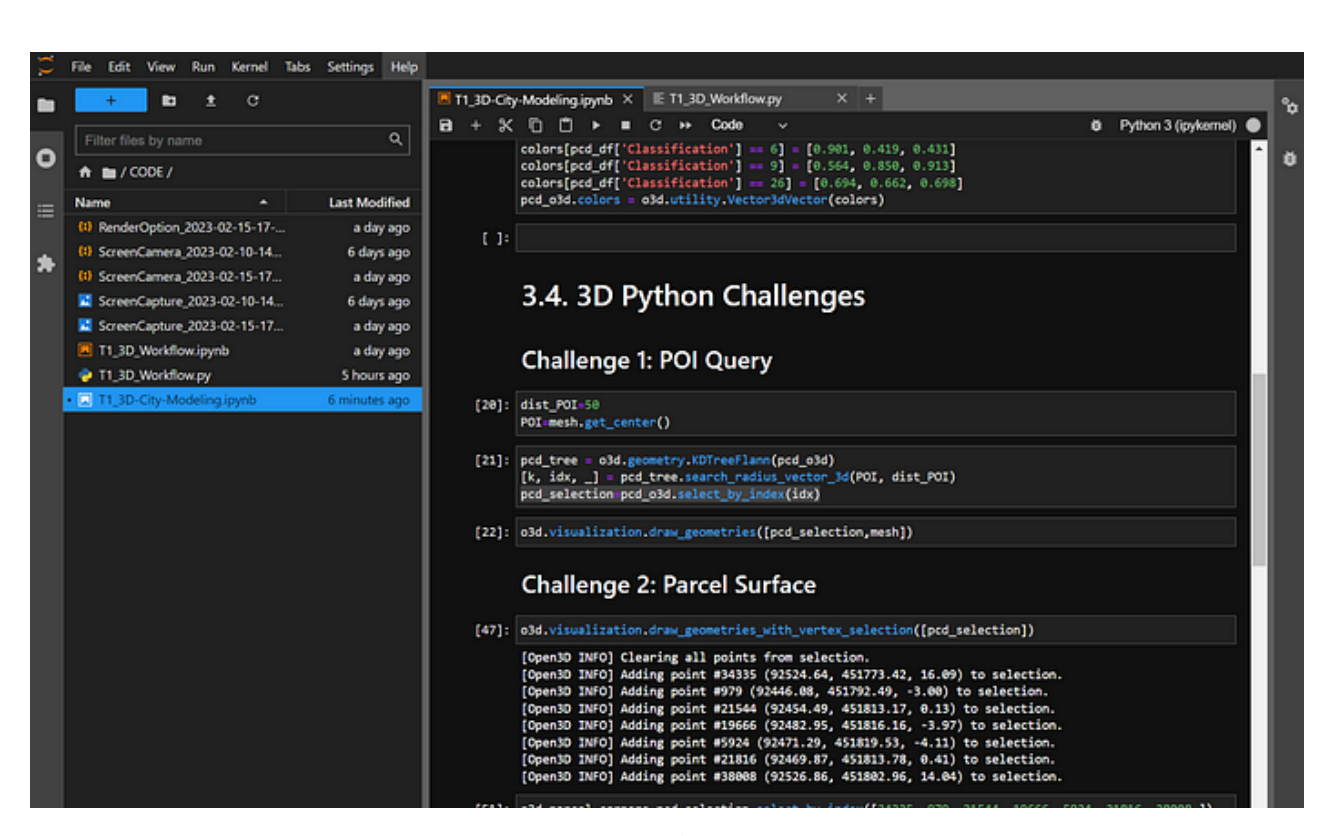
🦚 **Note**: *For this tutorial, we chose a Python version of 3.9.16, as shown above. Once you created an environment and after clicking on it, you can see a "play" icon next to its name. This opens the possibility of launching an* ***Anaconda Terminal*** *directly acting in the selected environment.* ***This is the go-to way to install libraries or IDEs in this environment***.

If you have a working Anaconda Navigator and a new environment created, we are ready to choose a way to code, intending to be more efficient than a text editor. 😁

### 1.2. The Python IDE

A Python IDE (Integrated Development Environment) is a software application that provides a comprehensive set of tools for developing, testing, and debugging Python code. They offer an all-in-one environment where we can write, edit, and execute code, manage project files, track changes, and collaborate with other developers. Some popular Desktop Python IDEs include PyCharm, Visual Studio Code, and Spyder, each offering unique features and capabilities to suit different programming needs.

Today, I want to highlight a great "web-based" IDE: JupyterLab. One of the main benefits of JupyterLab is its notebook interface, which provides a visual, interactive environment for working with code. It makes it easy to create, edit, and run code cells in real time without switching between different windows or applications. Additionally, JupyterLab supports a wide range of data visualization tools, making it an excellent choice for working with any geodata science or 3D machine learning project.


File Edit View Run Kernel Tabs Settings Help
/ CODE /
Name
Last Modified
RenderOption_2023-02-15-17...
ScreenCamera_2023-02-10-14...
ScreenCamera_2023-02-15-17...
ScreenCapture_2023-02-10-14...
ScreenCapture_2023-02-15-17...
T1_3D_Workflow.ipynb
T1_3D_Workflow.py
T1_3D-City-Modeling.ipynb
T1_3D-City-Modeling.ipynb
T1_3D_Workflow.py
Code
Python 3 (ipykernel)

```
colors[pcd_df['Classification'] == 6] = [0.901, 0.419, 0.431]
colors[pcd_df['Classification'] == 9] = [0.564, 0.850, 0.913]
colors[pcd_df['Classification'] == 26] = [0.694, 0.662, 0.698]
pcd_o3d.colors = o3d.utility.Vector3dVector(colors)
```

3.4. 3D Python Challenges

Challenge 1: POI Query

```
[20]: dist_POI=50
      POI=mesh.get_center()

[21]: pcd_tree = o3d.geometry.KDTreeFlann(pcd_o3d)
      [k, idx, _] = pcd_tree.search_radius_vector_3d(POI, dist_POI)
      pcd_selection=pcd_o3d.select_by_index(idx)

[22]: o3d.visualization.draw_geometries([pcd_selection,mesh])
```

Challenge 2: Parcel Surface

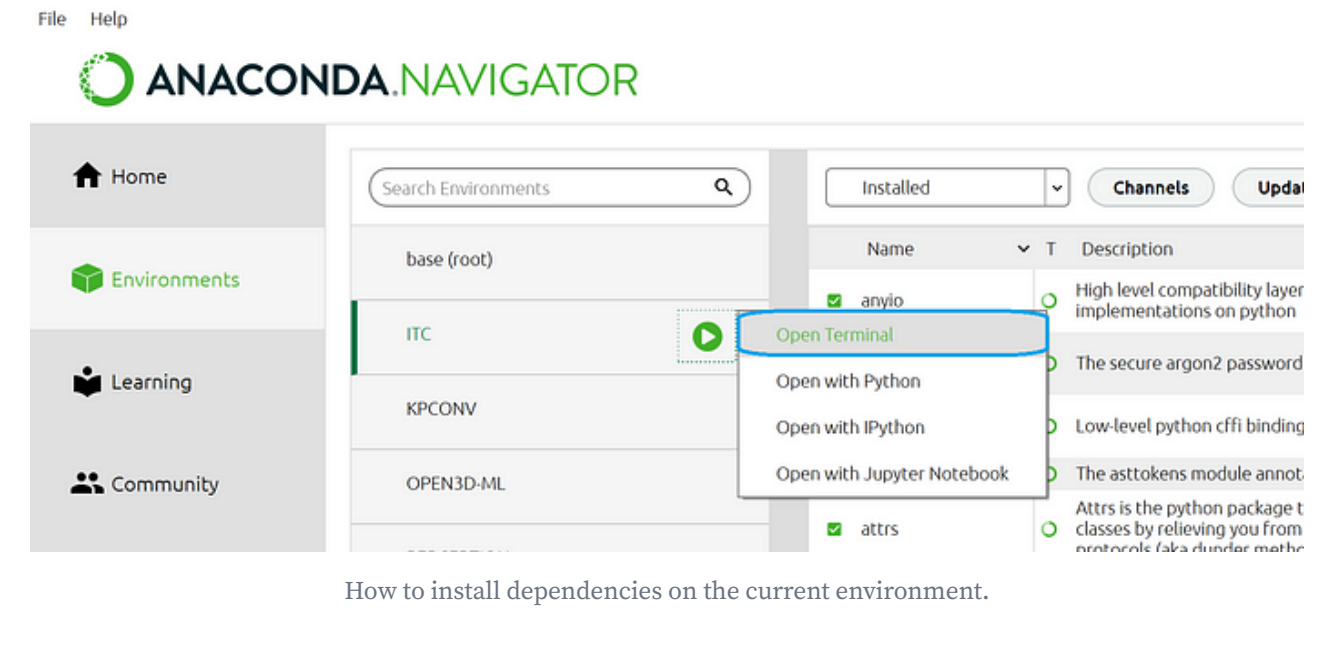
```
[47]: o3d.visualization.draw_geometries_with_vertex_selection([pcd_selection])

[Open3D INFO] Clearing all points from selection.
[Open3D INFO] Adding point #34335 (92524.64, 451773.42, 16.09) to selection.
[Open3D INFO] Adding point #979 (92446.08, 451792.49, -3.00) to selection.
[Open3D INFO] Adding point #21544 (92454.49, 451813.17, 0.13) to selection.
[Open3D INFO] Adding point #19666 (92482.95, 451816.16, -3.97) to selection.
[Open3D INFO] Adding point #5924 (92471.29, 451819.53, -4.11) to selection.
[Open3D INFO] Adding point #21816 (92469.87, 451813.78, 0.41) to selection.
[Open3D INFO] Adding point #38008 (92526.86, 451802.96, 14.04) to selection.
```

JupyterLab IDE for 3D Python.

JupyterLab's intuitive interface, robust feature set, and support for multiple programming languages make it a popular choice for Python developers of all skill levels.

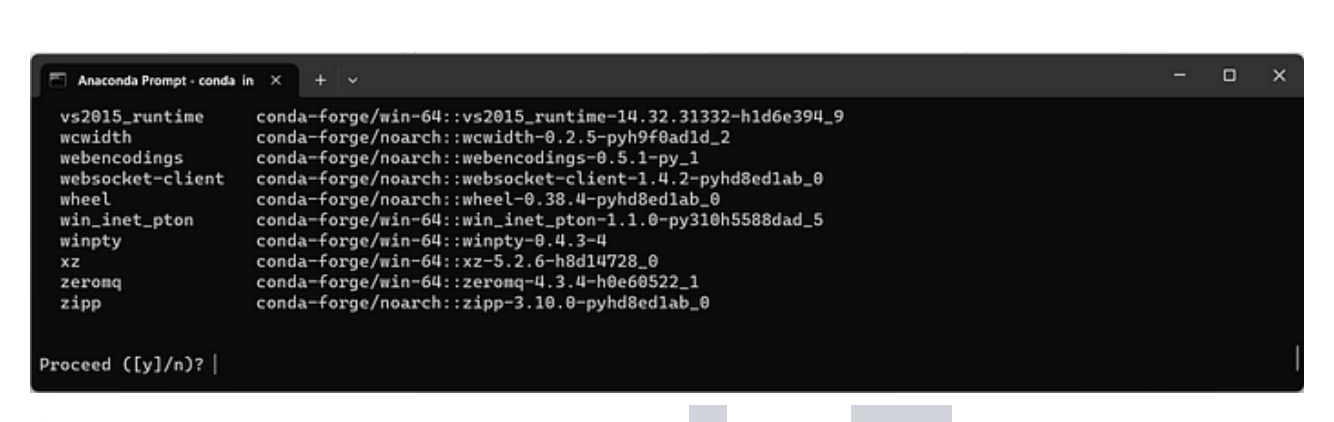
🦚 **Note**: *JupyterLab IDE also offers support for multiple programming languages, including Python, R, and Julia, allowing one to work with various tools and libraries within a single environment. And this is massively cool, as R and Julia are lovely languages.*

Before being able to use JupyterLab, we need to install it in our current Anaconda Environment. As mentioned before, we have to open the Anaconda Terminal in the environment of choice (ITC in our case). To achieve this with Anaconda Navigator, from your selected environment, (1) click on the green arrow, (2) select "`Open Terminal`" as shown below.


File Help
ANACONDA.NAVIGATOR
Home
Environments
Learning
Community
Search Environments
base (root)
ITC
KPCONV
OPEN3D-ML
Installed
Channels
Name
Description
anyio
High level compatibility layer for multiple asynchronous event loop implementations on python
The secure argon2 password
Low-level python cffi binding
The asttokens module annot
attrs
Attrs is the python package t classes by relieving you from
Open Terminal
Open with Python
Open with IPython
Open with Jupyter Notebook

How to install dependencies on the current environment.

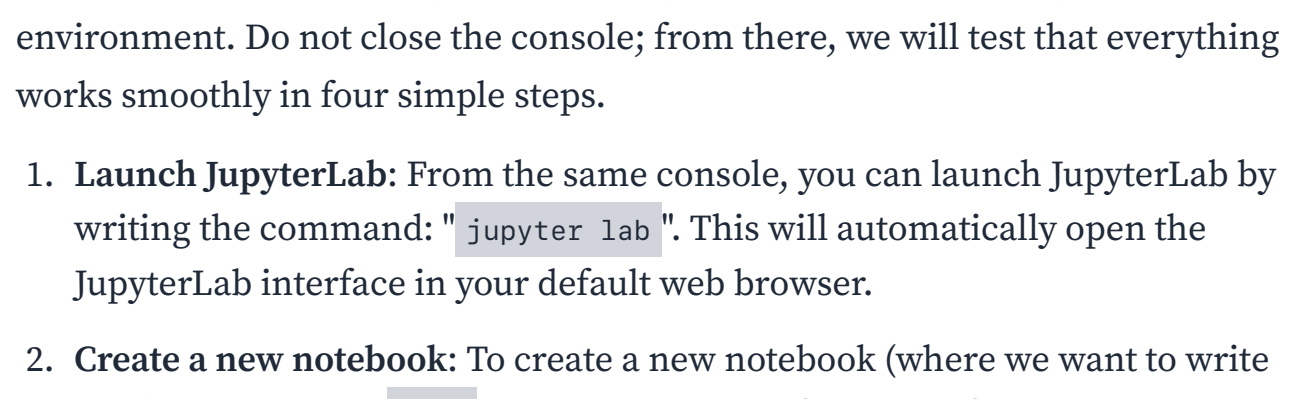
In the console (Anaconda Terminal) that opens, write the following line: `conda install -c conda-forge jupyterlab`, and press "`Enter`". This will install JupyterLab directly.



```
(ITC) C:\Users\flore>conda install -c conda-forge jupyterlab
```

The command line is executed in the Anaconda Terminal (we see that we are in the (ITC) environment).

Note that it may ask for your approval to install some needed library, which you need to accept by typing `y` followed by pressing the key "`Enter`".



```
vs2015_runtime     conda-forge/win-64::vs2015_runtime-14.32.31332-h1d6e394_9
wcwidth            conda-forge/noarch::wcwidth-0.2.5-pyh9f0ad1d_2
webencodings       conda-forge/noarch::webencodings-0.5.1-py_1
websocket-client   conda-forge/noarch::websocket-client-1.4.2-pyhd8ed1ab_0
wheel              conda-forge/noarch::wheel-0.38.4-pyhd8ed1ab_0
win_inet_pton      conda-forge/win-64::win_inet_pton-1.1.0-py310h5588dad_5
winpty             conda-forge/win-64::winpty-0.4.3-4
xz                 conda-forge/win-64::xz-5.2.6-h8d14728_0
zeromq             conda-forge/win-64::zeromq-4.3.4-h0e60522_1
zipp               conda-forge/noarch::zipp-3.10.0-pyhd8ed1ab_0

Proceed ([y]/n)?
```

After 30+ seconds, the installation needs our approval. type '`y`' and press `Enter`. This will download, extract and install the libraries mentioned above.

Once the process is done, you have a JupyterLab IDE installed in your ITC Conda environment. Do not close the console; from there, we will test that everything works smoothly in four simple steps.

1. **Launch JupyterLab**: From the same console, you can launch JupyterLab by writing the command: "`jupyter lab`". This will automatically open the JupyterLab interface in your default web browser.
2. **Create a new notebook**: To create a new notebook (where we want to write code), click on the "`File`" menu in the top-left corner of the JupyterLab interface and select "`New Notebook`." This will open a new notebook in a new tab.
3. **Write code**: You can now start writing code in the notebook. To create a new code cell, click the "`+`" button in the toolbar or press "`Ctrl + Shift + Enter`." You can then write your Python code (E.g. : `'This is working'` in the cell and run it by pressing "`Shift + Enter`."
4. **Save your work**: Remember to save your work regularly by clicking the "`Save`" button in the toolbar or using the "`Ctrl + S`" keyboard shortcut.

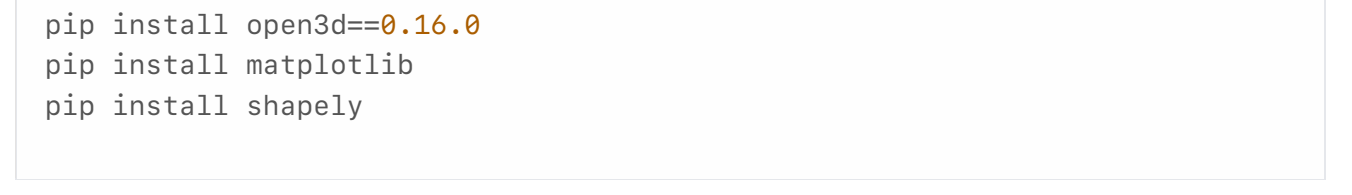
These are the basic steps to get started with JupyterLab. As you become more familiar with the interface, you can explore more advanced features, such as adding markdown text, importing and exporting data, and working with JupyterLab extensions.

🦚 **Note**: *For students at ITC — University of Twente, we have the luck to have the* ***CRIB: A Geospatial Computing Platform*** *using Jupyter Lab. I highly recommend using this cloud computing service if your computer shows some processing limitations while following this course.*

### 1.3. 3D Python Libraries

For this tutorial, I will introduce five libraries that are key to 3D Geospatial Analysis. These are `NumPy`, `Pandas`, `Open3D`, `Matplotlib,` and `Shapely`.

🦚 **Note**: *if you want to use the mentioned libraries, we must ensure they are installed and available in your environment. Therefore, in the same environment terminal, we use the formula "*`pip install package-name==version`*" (the* `==version` *is optional to fix certain versions) as follows:*



```
(ITC) C:\Users\flore>pip install open3d==0.16.0
```

Copy

```
#We install the 5 libraries with the package manager 'pip', one line at a ti
pip install numpy
pip install pandas
pip install open3d==0.16.0
pip install matplotlib
pip install shapely
```

**NumPy:** This library is used for working with arrays and matrices. It provides fast and efficient operations on large, multi-dimensional arrays, making it a powerful tool for scientific computing and data analysis. One hands-on example of how to use `NumPy` is to create a point cloud as a set of data points in the 3D euclidean space. To do this, you can create a NumPy array with three columns, where each row represents a single point in the point cloud:

Copy

```
import numpy as np
point_cloud = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

#To print the "point_cloud" variable as [Out] from the cell
print(point_cloud)
```

In this example, the point cloud has three points with coordinates `(1, 2, 3)`, `(4, 5, 6)`, and `(7, 8, 9)`, respectively. Easy, Peasy, you said? 😁

**Pandas:** This library is more geared towards data manipulation and analysis. It provides robust data structures and tools for working with structured data, such as CSV files, spreadsheets, and databases. While it's not specifically designed for 3D data processing, it can still be used to write and access point clouds in a tabular format. To do this, you can create a DataFrame object with columns representing the X, Y, and Z coordinates of each point:

Copy

```
import pandas as pd

# create a DataFrame with X, Y, and Z columns
points_df = pd.DataFrame({ 'X': [1, 4, 7], 'Y': [2, 5, 8], 'Z': [3, 6, 9]})
```

```python
# save the DataFrame as a CSV file at the same place as your script
points_df.to_csv('point_cloud.csv', index=False)
```

In this example, the point cloud has three points with coordinates `(1, 2, 3)`, `(4, 5, 6)`, and `(7, 8, 9)`, as obtained with NumPy. The DataFrame is then saved to a CSV file using the `to_csv` function.

🦚 **Note:** *Pandas can be extended with another Python module:* ***Geopandas***. *This library makes it possible to directly work with spatial data stored e.g. in Shapefiles or PostGIS database. This extends the scope of the current tutorial, but it is good to know because we will surely use it in other cases.* 😉

**Open3D:** This library is geared toward 3D data processing and visualization. It provides various tools and functions for working with point clouds, meshes, and other 3D data formats, such as voxels.

🦚 **Note:** *A quick way to install the library is to run the Anaconda Environment Terminal the same way as before and type the following command:* `pip install open3d==0.16.0`, *which will install the version 0.16.0 of Open3D, as used in this tutorial.*

Now, back in the Jupyter Lab IDE, let us create a point cloud using built-in Open3D functions:

```python
import open3d as o3d

# create an empty point cloud object
point_cloud = o3d.geometry.PointCloud()

# add points to the point cloud
points = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
point_cloud.points = o3d.utility.Vector3dVector(points)
```

In this example, the point cloud has the same three points with coordinates `(1, 2, 3)`, `(4, 5, 6)`, and `(7, 8, 9)`, respectively.

🦚 **Note:** *In the code block above, we created an Open3D PointCloud object and then* passed *the points list to the points attribute through the* `o3d.utility.Vector3dVector` *function that converts the list of points into a format that can be added to the point cloud object.*

**Matplotlib:** This library is used for data visualization. It provides many tools for creating high-quality charts, graphs, and other visualizations. While it's not specifically designed for 3D data visualization, it can still create 3D scatter plots of point clouds. To do this, you can use the Axes3D class to create a 3D plot and the scatter function to plot the points:

```python
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# create a point cloud with 1000 random points
points = np.random.rand(1000, 3)

# create a 3D plot and add the points as a scatter plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:,0], points[:,1], points[:,2])

# show the plot
plt.show()
```

In this example, a point cloud with 1000 random points is generated using NumPy. The points are then plotted as a scatter plot in a 3D plot using the `scatter` function. Finally, the plot is displayed using the `show` function.

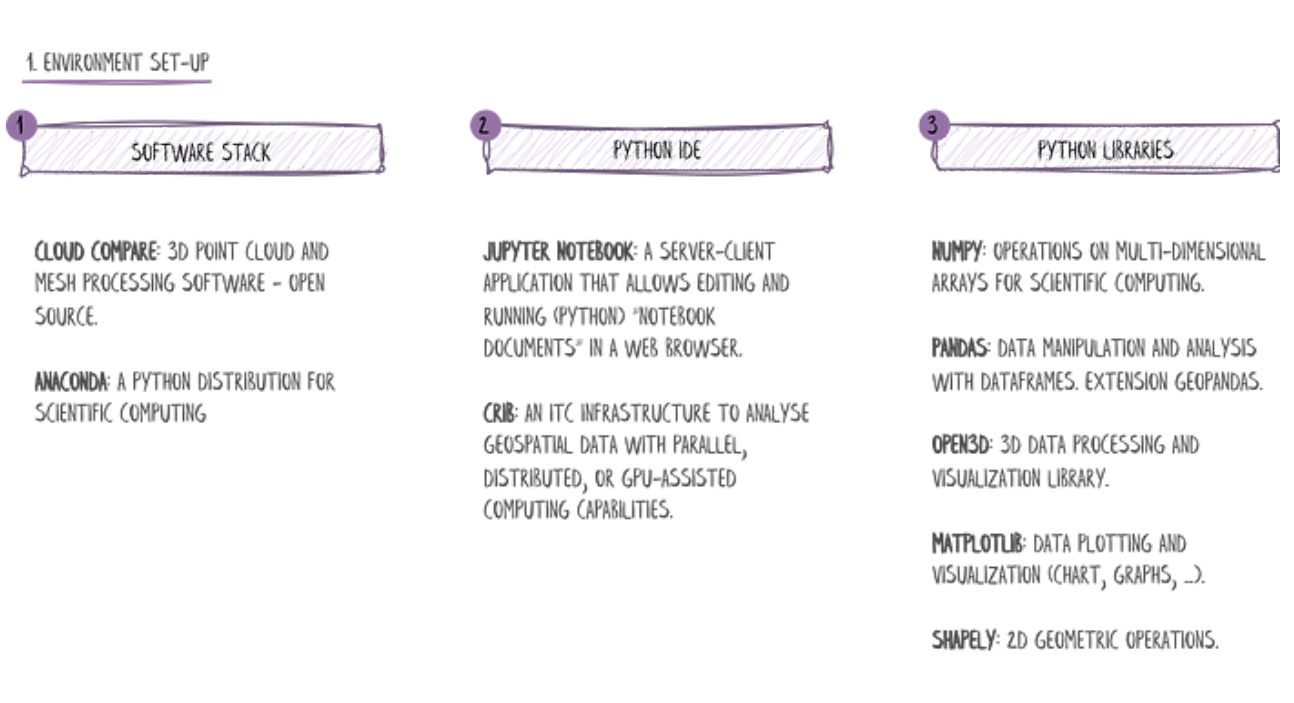
**Shapely:** This library is used mainly for 2D geometric operations to create, manipulate, and analyze geometric objects. It provides a wide range of tools for working with points, lines, polygons, and other geometric shapes. One common use case for `Shapely` is creating a polygon and checking if a point is within the polygon. To do this, you can create a Polygon object using a list of coordinates that define the polygon and then use the `contains` function to check if a point is within the polygon:

```python
from shapely.geometry import Point, Polygon

# create a polygon with four vertices
polygon = Polygon([(0, 0), (0, 1), (1, 1), (1, 0)])

# create a point
point = Point(0.5, 0.5)

# check if the point is within the polygon
if polygon.contains(point):
 print('The point is within the polygon.')
else:
 print('The point is outside the polygon.')
```

This is just a simple example, but Shapely provides many other functions for working with geometric objects that can be useful for more complex applications.

🦚 **Note:** *In this example, a polygon with four vertices is created using a list of coordinates. A point is then created using the* `Point` *function. Finally, the* `contains` *function is used to check if the point is within the polygon, and a message is printed to the console depending on the result of the check.*

Our first step in this 3D Workflow for 3D City Modeling is a clever combination of simple yet effective and proven tools and libraries, visually summarized below.

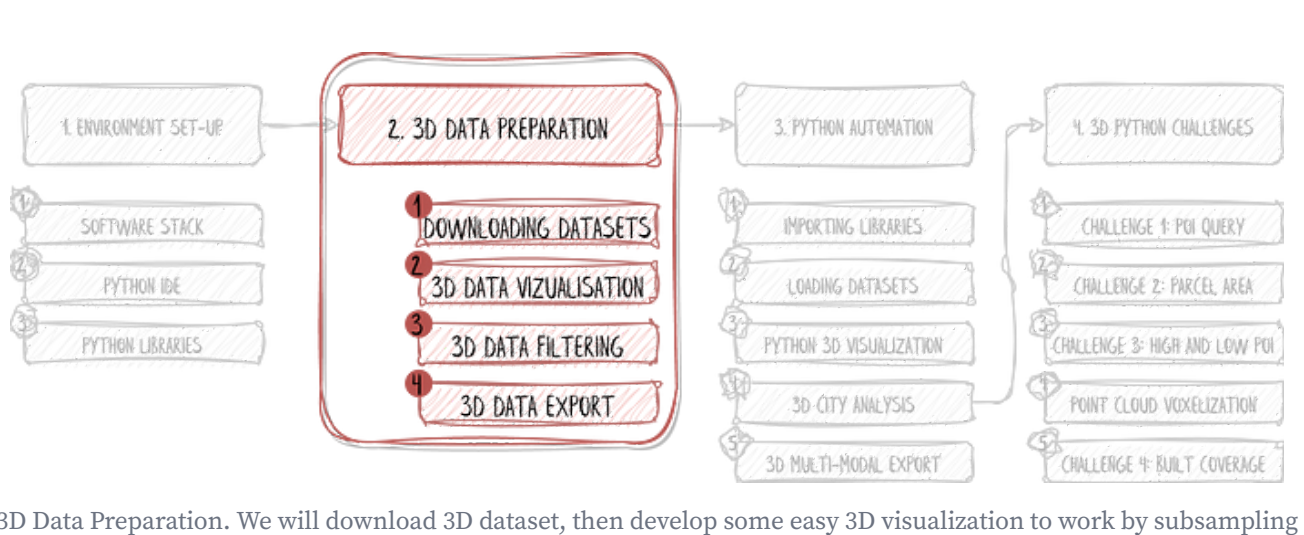

1. ENVIRONMENT SET-UP
1 SOFTWARE STACK
CLOUD COMPARE: 3D POINT CLOUD AND MESH PROCESSING SOFTWARE – OPEN SOURCE
ANACONDA: A PYTHON DISTRIBUTION FOR SCIENTIFIC COMPUTING
2 PYTHON IDE
JUPYTER NOTEBOOK: A SERVER-CLIENT APPLICATION THAT ALLOWS EDITING AND RUNNING (PYTHON) 'NOTEBOOK DOCUMENTS' IN A WEB BROWSER.
CRIB: AN ITC INFRASTRUCTURE TO ANALYSE GEOSPATIAL DATA WITH PARALLEL, DISTRIBUTED, OR GPU-ASSISTED COMPUTING CAPABILITIES.
3 PYTHON LIBRARIES
NUMPY: OPERATIONS ON MULTI-DIMENSIONAL ARRAYS FOR SCIENTIFIC COMPUTING.
PANDAS: DATA MANIPULATION AND ANALYSIS WITH DATAFRAMES. EXTENSION GEOPANDAS.
OPEN3D: 3D DATA PROCESSING AND VISUALIZATION LIBRARY.
MATPLOTLIB: DATA PLOTTING AND VISUALIZATION (CHART, GRAPHS, ...).
SHAPELY: 2D GEOMETRIC OPERATIONS.

The summary of the environment set-up. © Author

Now that we have an explicit software stack, a functional python IDE and a basic understanding of python libraries, we can start preparing our data.

## Step 2. 3D Data Preparation

We now move on to the second step: 3D Data Preparation.



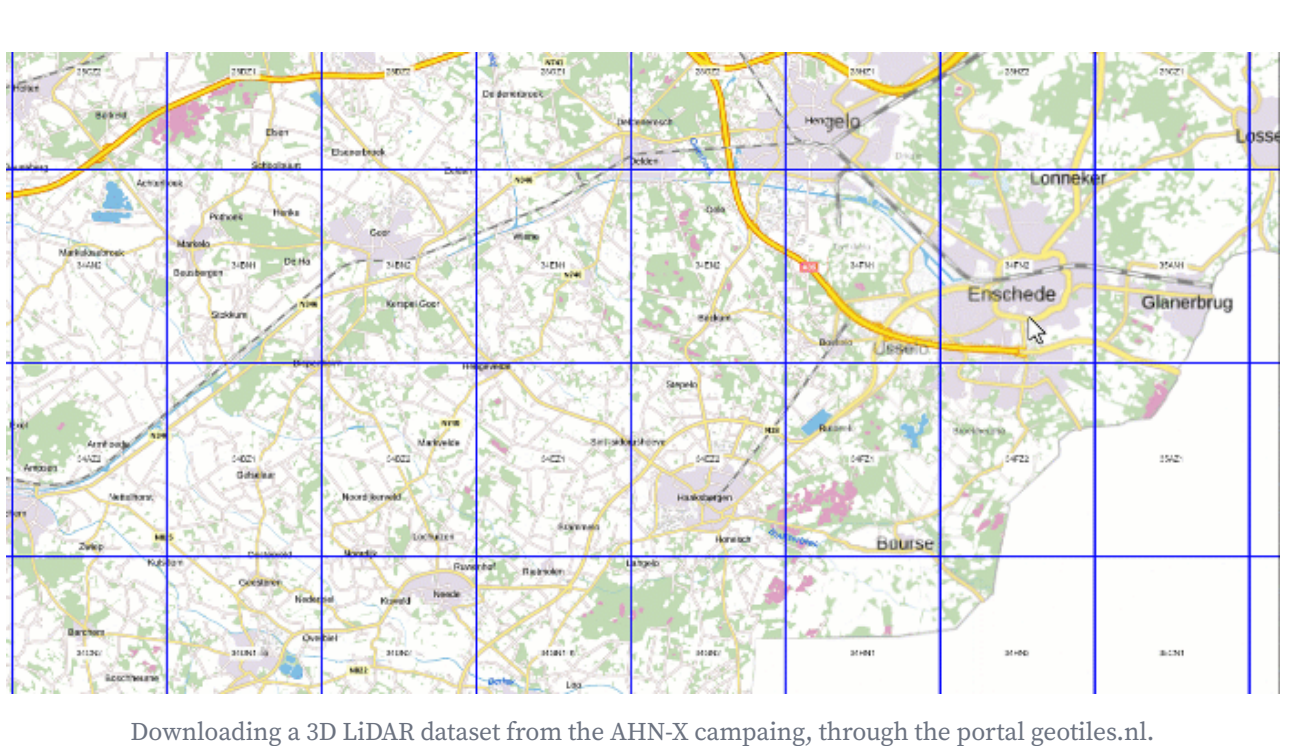
3D Data Preparation. We will download 3D dataset, then develop some easy 3D visualization to work by subsampling and exporting the data in a ready-to-use Python-friendly format.

Our goal is to gather and prepare the datasets to be usable for our analysis with Python in mind. Therefore, this step also acts as a "3D Data Visualization" phase, where we will qualitatively assess what we are dealing with. If you are ready, let us get started.

### 2.1. Downloading 3D Datasets

For 3D City Model Analysis, we gather some excellent datasets from OpenData Sources. I illustrate a specific tile of interest in the Netherlands, but I encourage you to study on your house or any point of interest for you (if you live in the Netherlands, of course, 😉).

**The Point Cloud Dataset.** First, we gather a point cloud dataset using the geotiles.nl portal, which provides some nice datasets under the CC-BY 4.0 license. For this, you can head over to the website, zoom in on the tile of interest, and get the latest version of the AHN LiDAR point cloud (in our case, AHN4), as illustrated below. The file will then be downloaded as a .laz (LasZip) file.



Downloading a 3D LiDAR dataset from the AHN-X campaing, through the portal geotiles.nl.

🦚 **Note:** *The AHN (Actueel Hoogtebestand Nederland) stems from an Aerial LiDAR coverage that provides a digital elevation map for all of the Netherlands. It contains detailed and precise height data with an average of eight measurements per square meter. Organizations such as the water boards, the provinces, and the Department of Public Works use the AHN for water and dam management. Based on the height and elevation of the ground level, it is determined whether the water can flow sufficiently from the land, how high the water level in the ditches can be, whether the water in rivers, flood plains, and ditches can be drained sufficiently and whether the dikes are still high and strong enough. The AHN is also used for many other types of management, such as daily management and maintenance of dikes, preparing specifications for significant maintenance, 3D mapping, permitting, and enforcement. Municipalities, businesses, and researchers also use detailed elevation data.*

If you like figures, I have a quick overview of AHN LiDAR data for you:

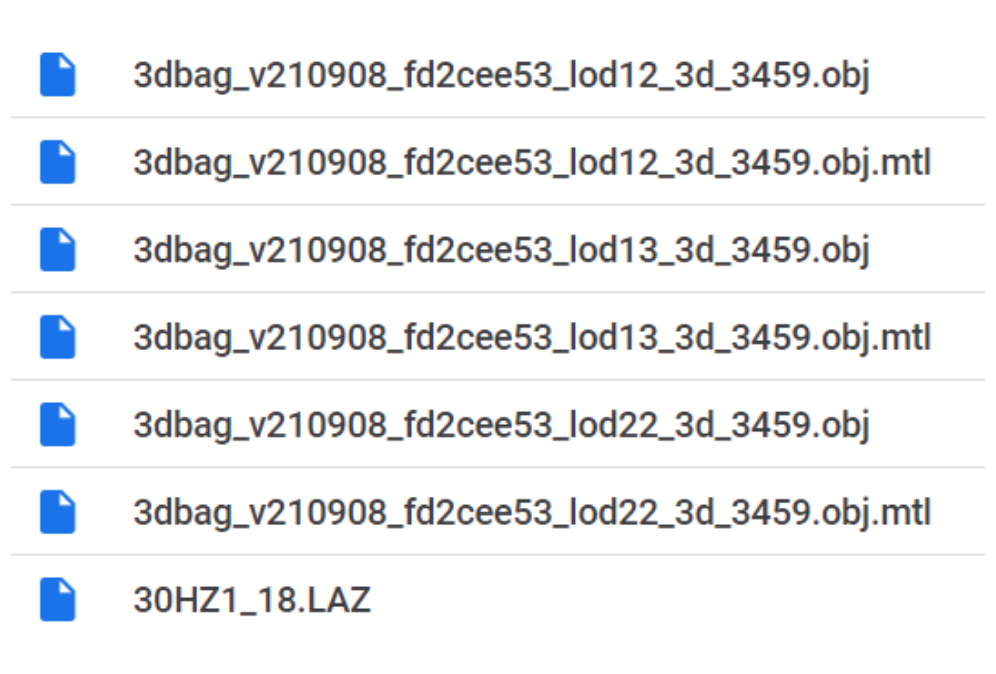
- AHN1: 1997–2004, 1 pt/16 m2 to 1 pt/m2
- AHN2: 2007–2012, 8 pts/m2
- AHN3: 2014–2019, 8 pts/m2
- AHN4: 2020–2022, 10–15 pts/m2,
- AHN5: 2023–2025, 10–15 pts/m2

**The Mesh Dataset.** For this part, we want to find a 3D Mesh in the same spot as the LiDAR area we downloaded. We thus use the platform 3DBAG created by TUDelft that allows us to retrieve 10M buildings in the Netherlands in LoD1.2, LoD1.3, and LoD2.2 from the CityGML Specification. At this step, we are mainly interested in the geometry. Still, we know that the semantics and topology are crucial aspects of the CityGML / City JSON Data Models and will be explored in further articles. 😉

Downloading a dataset from the portal 3Dbag.nl.

🦚 **Note:** *3DBAG contains 3D models at multiple levels of detail, which are generated by combining two available data sets: the building data from the BAG and the height data from the AHN. The 3D BAG is updated regularly, keeping it up-to-date with the latest openly available building stock and elevation information.*

After downloading the datasets, you should have one point cloud in the .laz file format and one or more .obj datasets (with their accompanying .mtl files) that describe approximately the same extent with different Levels of Detail (in our case LoD 1.2, 1.3, and 2.2), as illustrated below.



3dbag_v210908_fd2cee53_lod12_3d_3459.obj
3dbag_v210908_fd2cee53_lod12_3d_3459.obj.mtl
3dbag_v210908_fd2cee53_lod13_3d_3459.obj
3dbag_v210908_fd2cee53_lod13_3d_3459.obj.mtl
3dbag_v210908_fd2cee53_lod22_3d_3459.obj
3dbag_v210908_fd2cee53_lod22_3d_3459.obj.mtl
30HZ1_18.LAZ

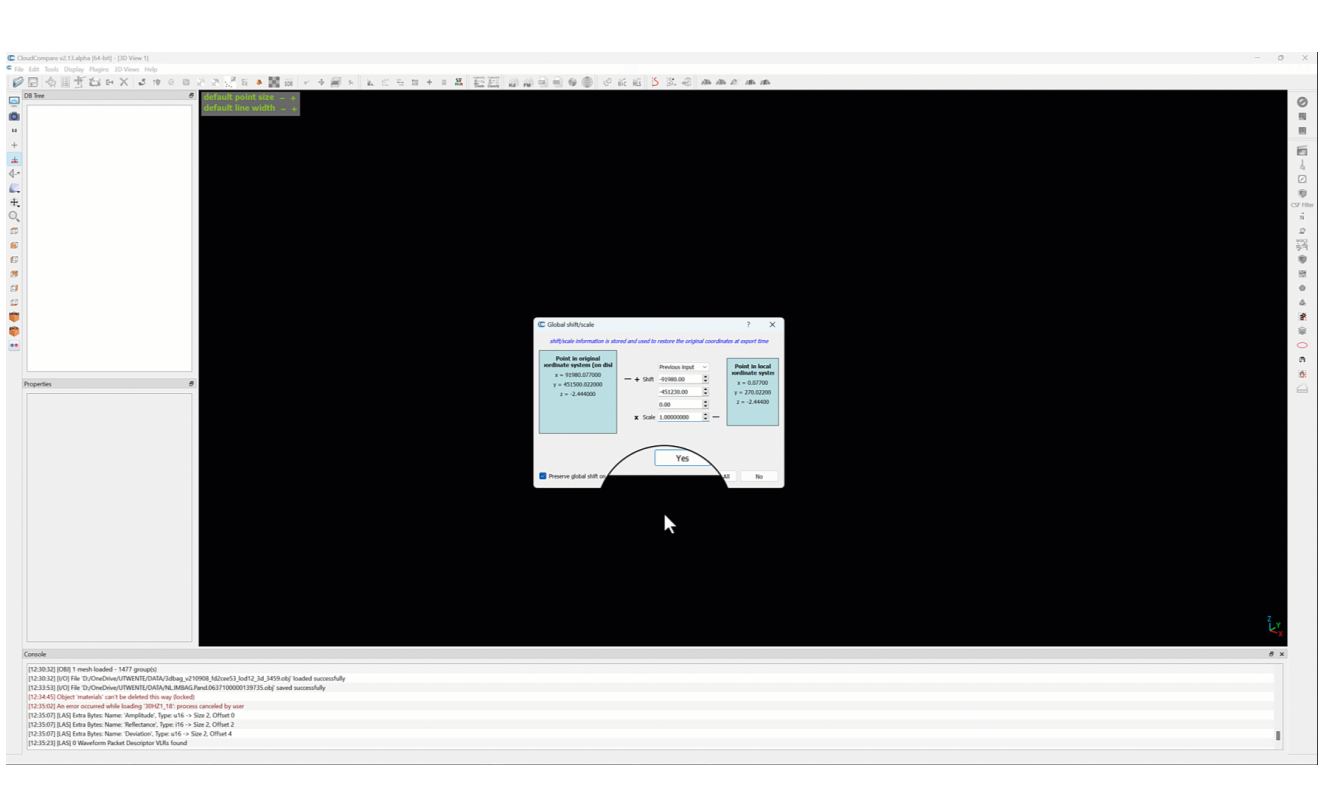
🧙 **Wizard:** [OPTIONAL] *If you want to deepen your expertise on 3D data file format, especially meshes from point clouds, I encourage you to follow the tutorial below that will show you in-depth how to mesh point clouds and how they are structured.*

**5-Step Guide to generate 3D meshes from point clouds with Python**
Tutorial to generate 3D meshes (.obj, .ply, .stl, .gltf) automatically from 3D point clouds using python. (Bonus)...
towardsdatascience.com

For the sake of convenience, you can directly download the selection from this **Drive Folder**.

### 2.2. 3D Data Visualization

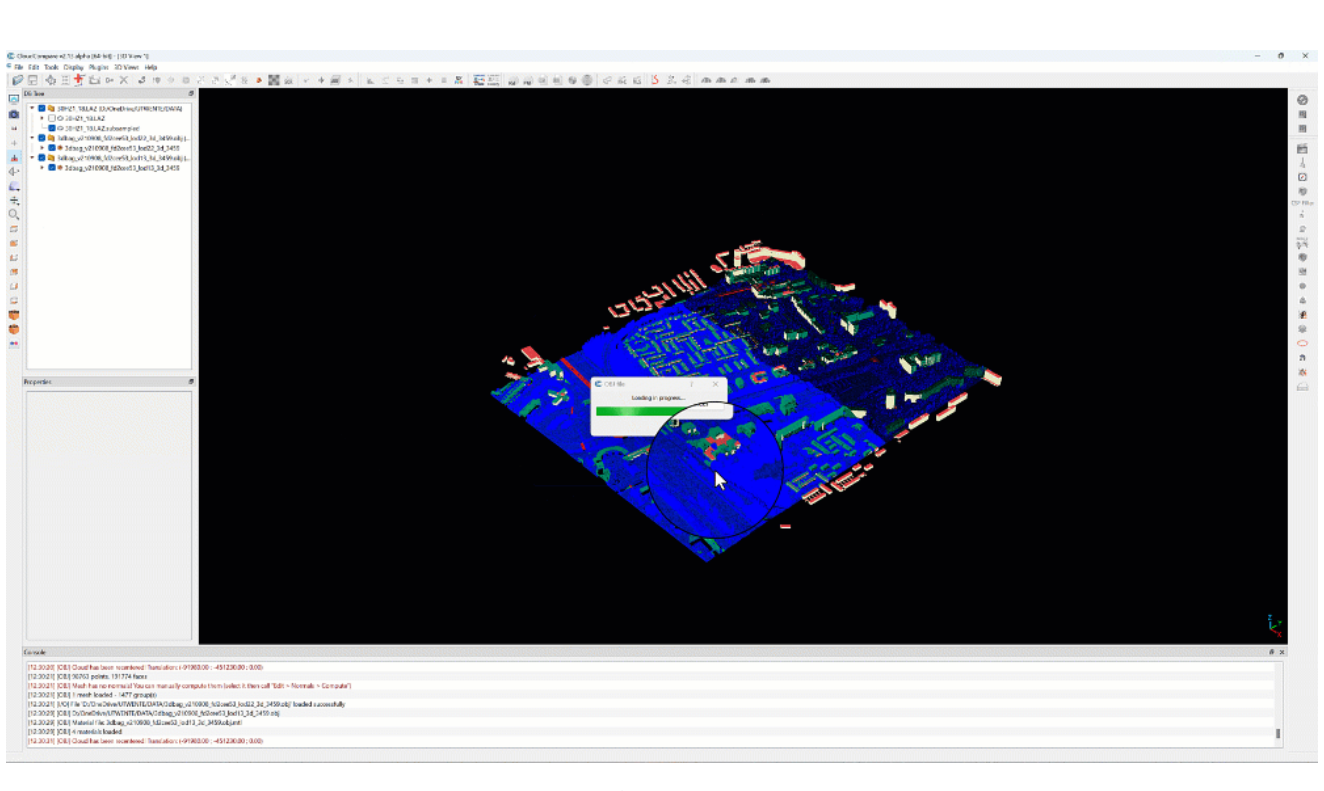
We will now jump into `CloudCompare` to ensure that the data downloaded for the analysis hold no distinctive surprise. 😁 After launching `CloudCompare`, you can load the `.laz` point cloud from your local folder. When the import window displays, ensure to import only the `"Classification"` Extra field for this application and accept the global shift as shown below.



3D Data Visualization within CloudCompare.

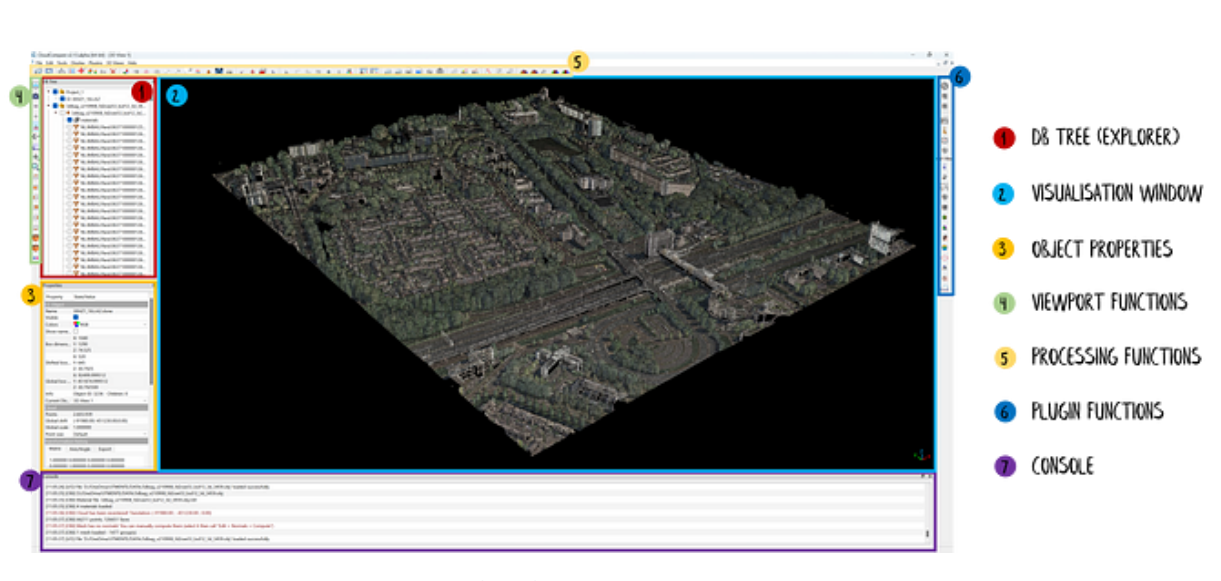
🦚 **Note:** *The Global Shift is a temporary shift to permit* `CloudCompare` *to work with georeferenced data that cross the bounds of the number of numbers it can handle for visualization purposes. Thus, it is a transparent step* and will be applied to the data upon saving.

We can now move on to importing the mesh data as well. For this, we execute the same import action and accept the proposed translation shift while ensuring it is the same as the one applied to the point cloud.



Loading 3D meshes from the downloaded assets.

You can now see from the Database Tree the different objects imported in your current project (You can check the image below if you cannot find the DBTree). The `DBTree` behaves a bit like your OS Explorer, where you have a folder that holds different point clouds or meshes. Each object (e.g., a point cloud or a 3D Mesh) can be activated ✅ (or deactivated) visually like a layer and selected to identify object properties.

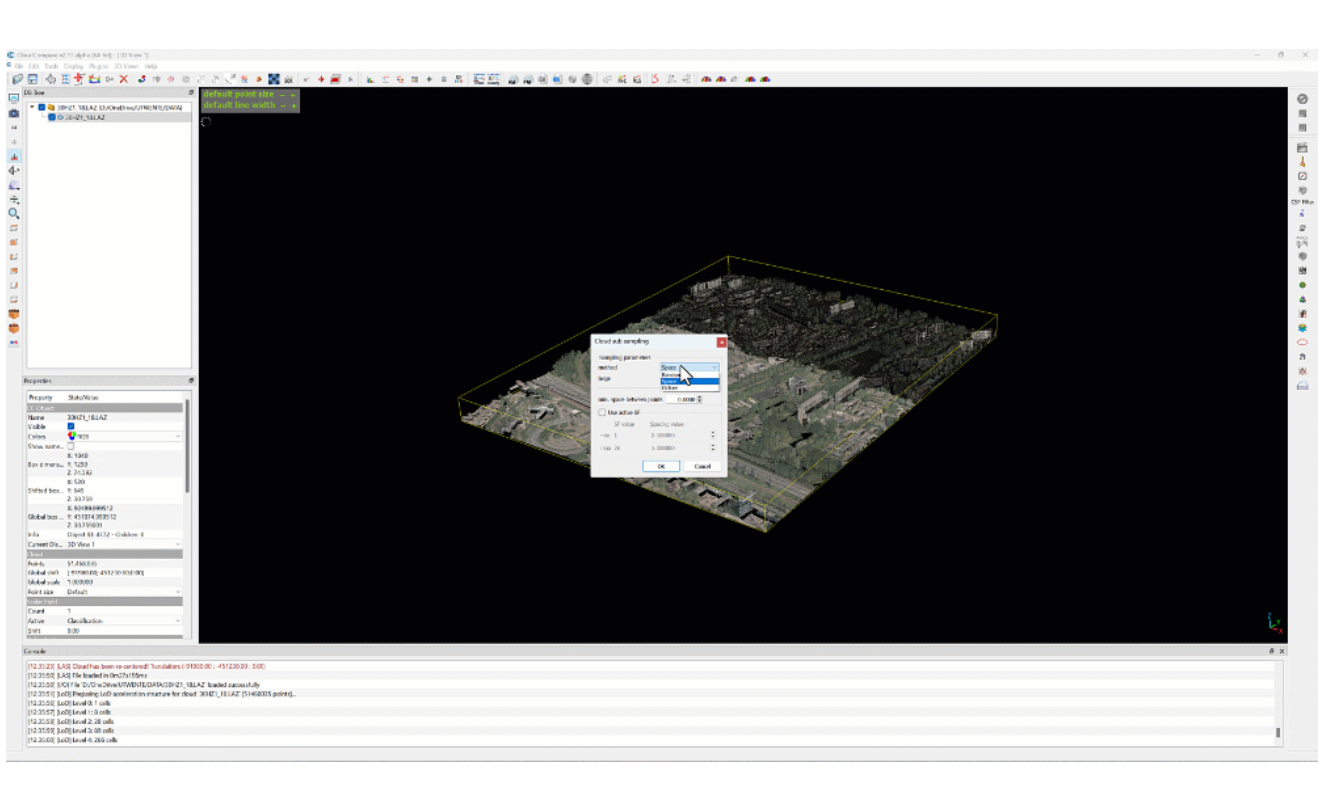


CloudCompare Interface for 3D data processing and analysis. © Author

🦚 **Note:** *CloudCompare does not save by default. To bypass any crash, if you are worried, you can always put all your data and analysis in one folder from the DBTree (Right-Click > Create New Empty Group) and save this folder as a .bin CloudCompare project.*

### 2.3. 3D Data Subsampling

It is now time to dive into 3D data filtering. First, we select the point cloud from the `DBTree` and apply a spatial subsampling function to keep one point every 50 cm, allowing us to retain sufficient information while not comprising the computational speed after that. For this, we use the `subsample` function as illustrated below.



3D Point Cloud Sub-sampling

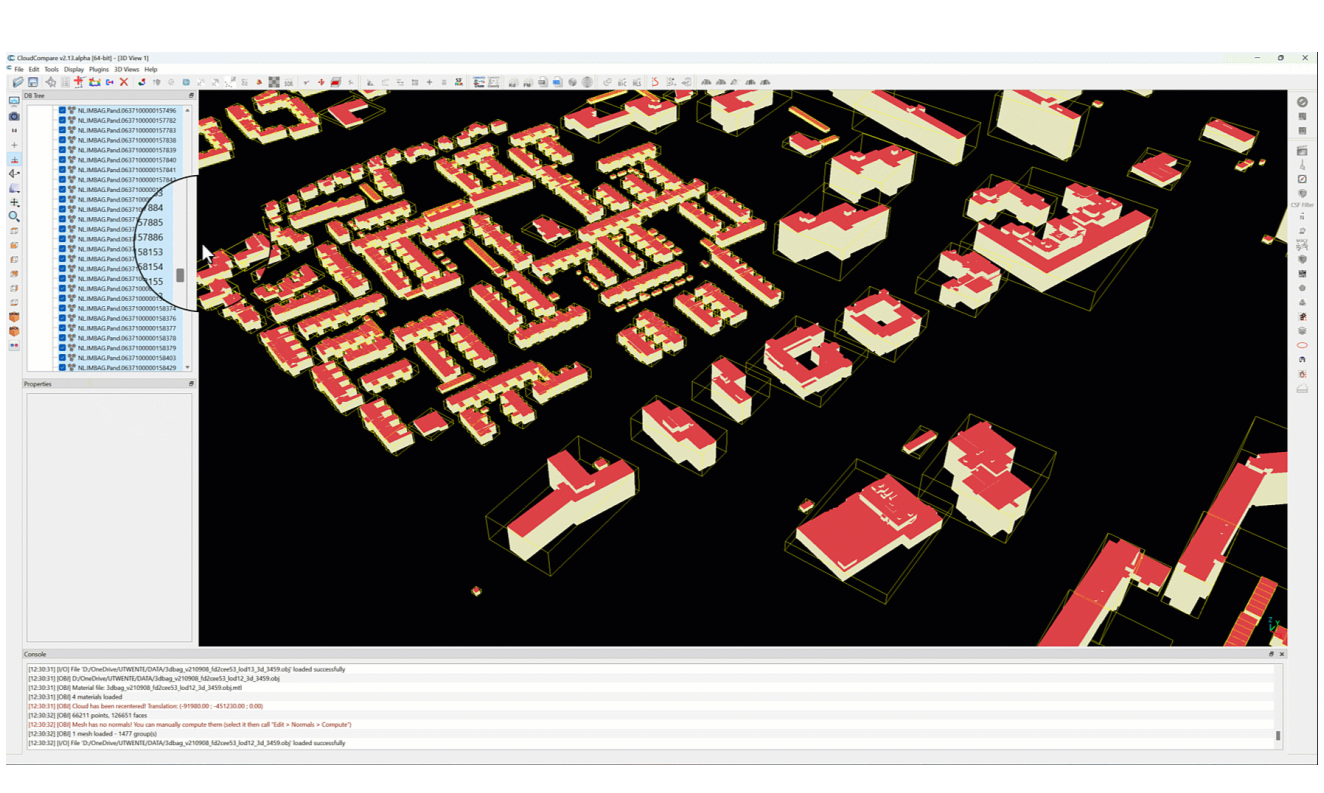
🧙 **Wizard:** *[OPTIONAL] In our case, we used a spatial subsampling function to retain one point every 50 cm on average. If you want to explore and deepen the 3D point cloud sampling strategies, I recommend diving into the following article.*

**How to automate LiDAR point cloud processing with Python**
The ultimate guide on point cloud sub-sampling from scratch, with Python. It covers LiDAR I/O, 3D voxel grid processing...
towardsdatascience.com

Once the subsampling step is done, we get a resulting subsampled point cloud in the `DBTree`, that we can use for downstream processes.

On the Mesh side, upon import, we see that we can open each mesh object from the DBTree by clicking on the little arrow icon next to it, which shows many different sub-mesh elements. This is because, in the original `.obj` file, we have an "object" refinement that permits us to retain some "semantic" decomposition linked to geometries. Each sub-mesh is a building entity decomposition.

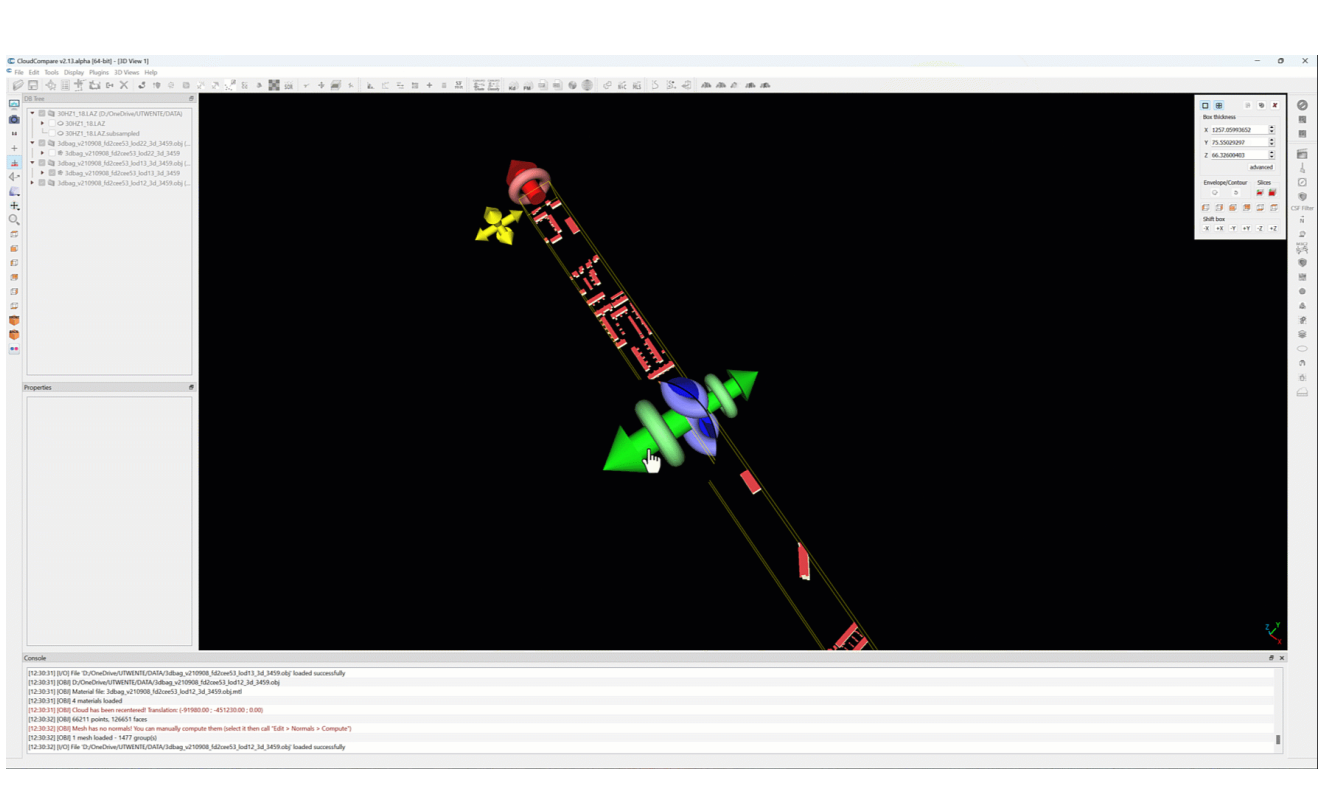
If you want to see these elements, you can open the mesh, select all the sub-element by holding `Ctrl+Shift` on the second mouse click, and `right-click > toggle` to display them. You can also uncheck the `Visible` property of the parent mesh element to ensure what you view is only the sub-elements, as shown below.



3D Data Preparation within CloudCompare.

🦚 **Note:** *We cannot use the sub-mesh selection as they also encompass all the vertices from the mesh parent, which means that, if we were to use it, we would have to segment the mesh to only the vertices of the sub-mesh selected.*

Excellent, well done! From there, you can uncheck all elements except the mesh you want to consider (in our case, the LoD 1.2) and use, E.g., the "Cross Section" function, to select the house you are interested in, as shown below.

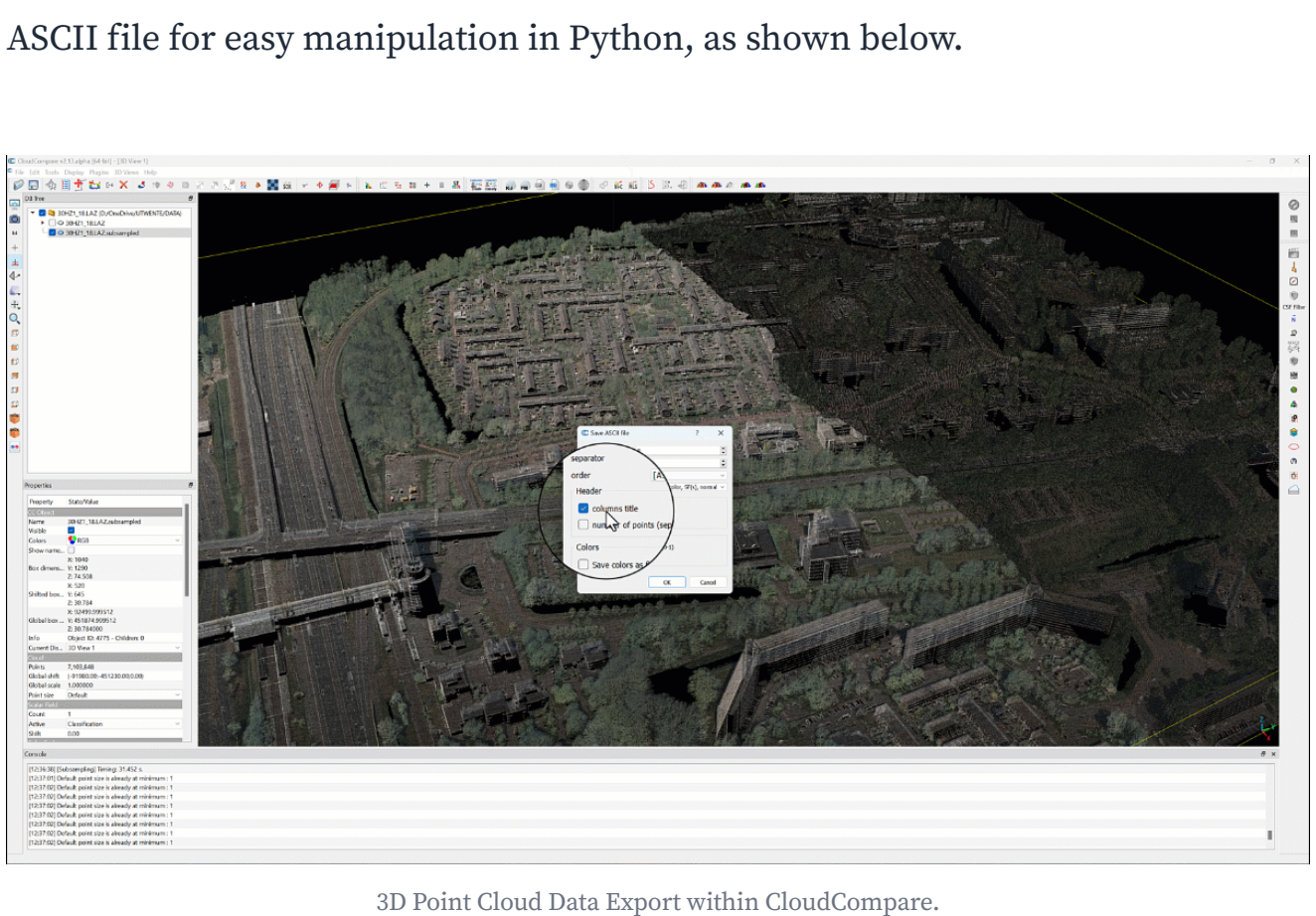


3D Data Selection within CloudCompare

Once this is done, we are ready to export our selected study site for both 3D modalities.

### 2.4. 3D Data Export

Concerning the point cloud data, we can export it in various file formats. Because we want to retain the classification field for some Python analysis (E.g., to know if a point belongs to the ground or a building), we export the 3D point cloud as an ASCII file for easy manipulation in Python, as shown below.

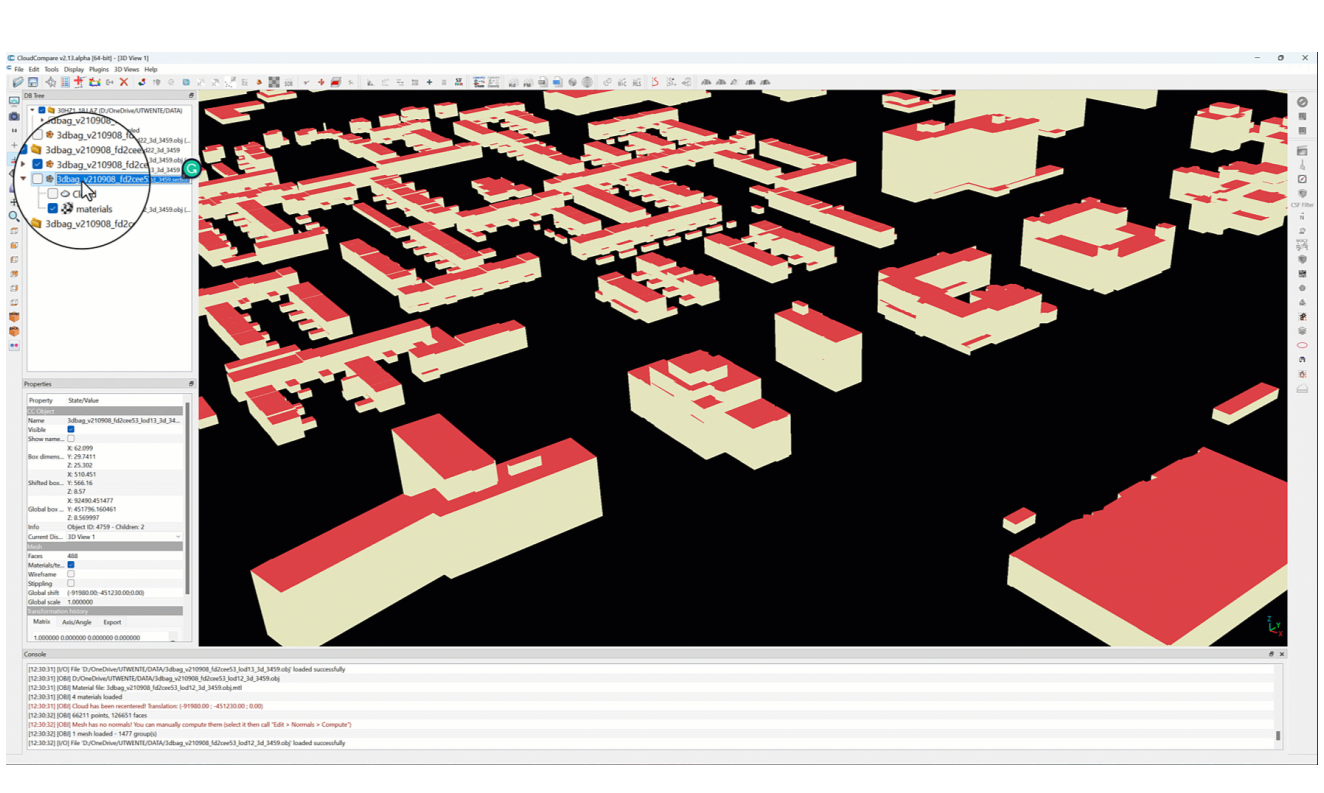


3D Point Cloud Data Export within CloudCompare.

🦚 **Note:** *When saving the file, we can optionally modify the file extension to* `.xyz` *and check the "keep the column names" option when the export dialog opens. This will allow having the column's name written in your file. However, if you open your file with a text editor, make sure to delete the two backslashes that are not really useful, as shown above.*

Our first 3D dataset is prepared and ready to be used as a `.xyz` file for Python. Now, let us move on to the 3D Mesh.

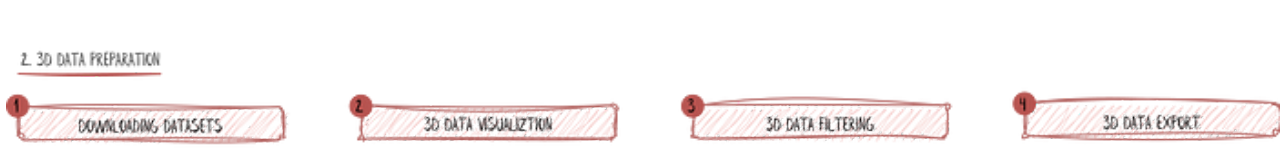
After selecting your house/building block of interest, I encourage you to get the associated sub-mesh element name and use this as the name for your exported file. Concerning the export dialog options, you can choose the .obj file extension, which is a safe bet 😉.



3D Mesh Export within CloudCompare.

The 3D Mesh is now ready and accompanied by a material `.mtl` file (not helpful in our case).

Step 2 is Complete! Well done. We first gathered a 3D Point Cloud and a 3D Mesh of a zone of the Netherlands. We then visualized them to check if they were corresponding to our intent. Then we filtered the point cloud to retain around 1 point every 50 cm, the 3D Mesh to keep only one object representing a building house, and we exported both modalities respectively as a `.xyz`, and a `.obj` + `.mtl` file.[1]



POINT CLOUD: DOWNLOADING FROM OPENDATA PORTAL AN AERIAL LIDAR AS A LAZ FILE
3D MESH: DOWNLOADING FROM OPENDATA PORTAL A 3D CITY MODEL AS AN .OBJ FILE
IMPORT POINT CLOUD: USING CLOUDCOMPARE, PROPERLY IMPORT POINT CLOUDS.
IMPORT MESH: USING CLOUDCOMPARE, PROPERLY IMPORT MESHES.
3D VISUALIZATION: MAKING SURE THE DATA PROPERLY ALIGNS AND IS REPRESENTATIVE ENOUGH FOR OUR GOAL(S).
SUBSAMPLING: OPERATIONS TO REDUCE THE NUMBER OF DATA POINTS FOR FURTHER ITERATIVE ANALYSIS, BEFORE SCALING.
SELECTION: REDUCING THE EXTENT OF THE ANALYSIS FOR THE OBJECT(S), SCENE(S) OF INTEREST, AND MAKING SURE THE DECOMPOSITION IS SOUND.
POINT CLOUD: EXPORTING THE POINT CLOUD AS AN ASCII FILE (E.G. .XYZ) FOR GENERAL ANALYSIS IN PYTHON
MESH: EXPORTING THE MESH AS AN .OBJ + ASSOCIATED .MTL FOR PYTHON ANALYSIS

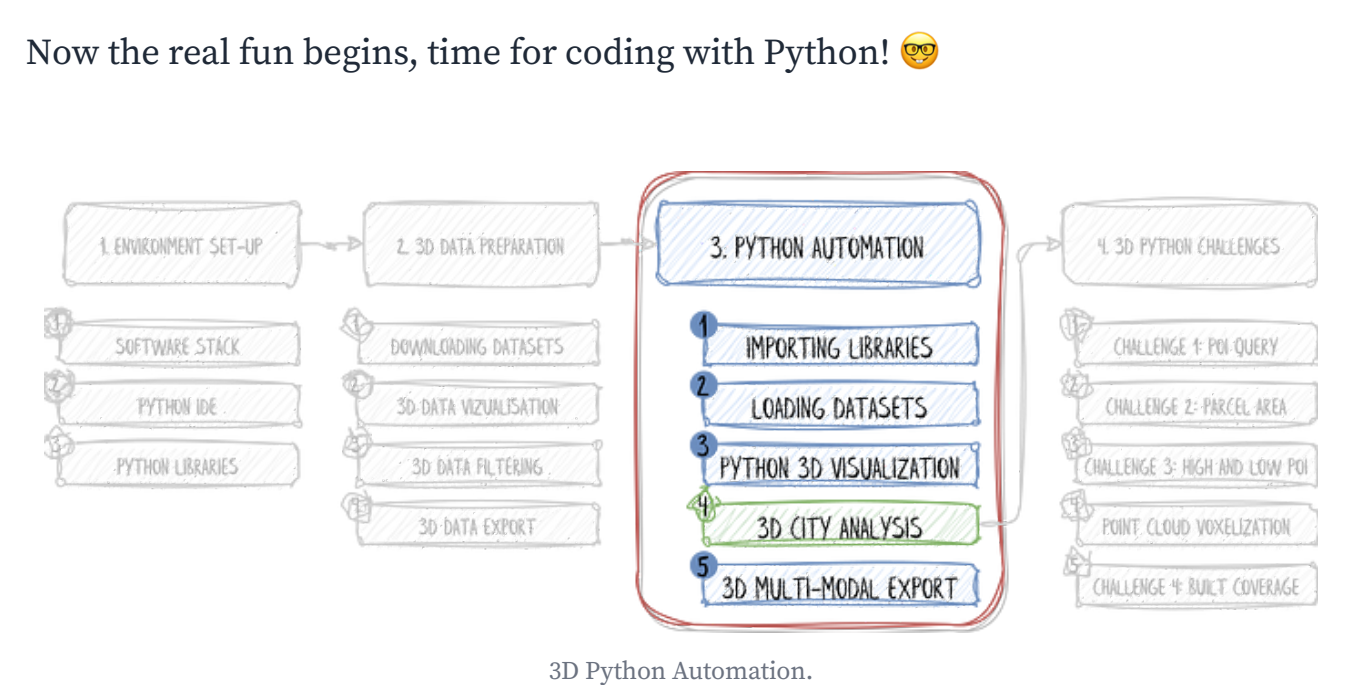
A visual summary of Step 2: 3D Data Preparation. © F. Poux

Let us now put these bad boys into a great Python setup to maximize automation and 3D analysis!

> [1]For convenience, you can find these datasets in this **Drive Folder**.

## Step 3. Python Automation

Now the real fun begins, time for coding with Python! 🤓


1. ENVIRONMENT SET-UP
SOFTWARE STACK
PYTHON IDE
PYTHON LIBRARIES
2. 3D DATA PREPARATION
DOWNLOADING DATASETS
3D DATA VIZUALISATION
3D DATA FILTERING
3D DATA EXPORT
3. PYTHON AUTOMATION
1 IMPORTING LIBRARIES
2 LOADING DATASETS
3 PYTHON 3D VISUALIZATION
4 3D CITY ANALYSIS
5 3D MULTI-MODAL EXPORT
4. 3D PYTHON CHALLENGES
CHALLENGE 1: POI QUERY
CHALLENGE 2: PARCEL AREA
CHALLENGE 3: HIGH AND LOW POI
POINT CLOUD VOXELIZATION
CHALLENGE 4: BUILT COVERAGE

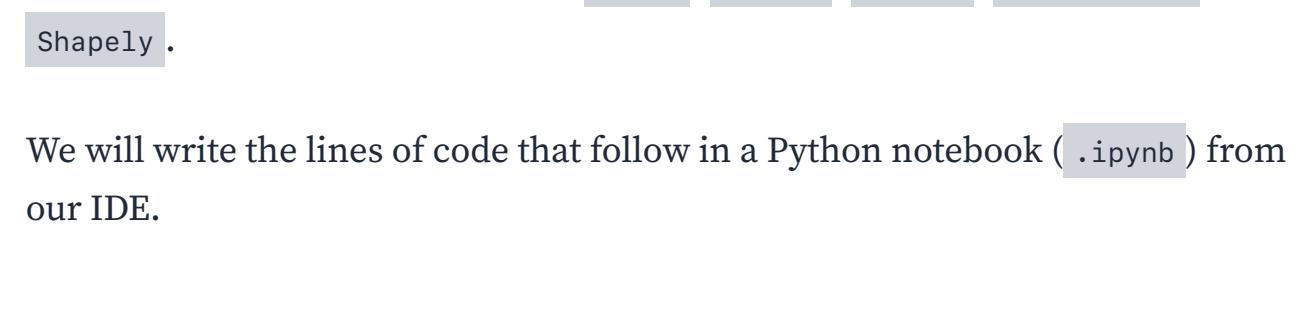
3D Python Automation.

As shown above, we will follow a five-stage approach by importing libraries, loading datasets, setting up our 3D Python Visualiser, Defining solutions to 3D Challenges, and then exporting our results to be used outside Python. Let us first build the bulk of our automated pipeline in Python before moving to the different challenges

### 3.1. Importing libraries

As defined in Step 1., we will stick to a minimal amount of libraries to deepen our expertise in their usage. These are `NumPy`, `Pandas`, `Open3D`, `Matplotlib,` and `Shapely`.

We will write the lines of code that follow in a Python notebook (`.ipynb`) from our IDE.


File
Edit
View
Run
Kernel
Tabs
Settings
Help
Filter files by name
/ CODE /
Name
Last Modified
RenderOption_2023-02-15-17-...
18 hours ago
ScreenCamera_2023-02-10-14...
6 days ago
ScreenCamera_2023-02-15-17...
18 hours ago
ScreenCapture_2023-02-10-14...
6 days ago
ScreenCapture_2023-02-15-17...
18 hours ago
T1_3D_Workflow.ipynb
a day ago
T1_3D_Workflow.py
18 hours ago
T1_3D-City-Modeling.ipynb
a minute ago
T1_3D-City-Modeling.ipynb
Code
Python 3 (ipykernel)

```
[ ]: #Here you type your Code :)
```

The IDE view to write our script.

We import the libraries mentioned above with the following code block:

```
import numpy as np
import pandas as pd
import open3d as o3d
import matplotlib.pyplot as plt
from shapely.geometry import Polygon

print(f"Open 3D Version: {o3d.__version__}")
```

This will return the current Open 3D Version as a string: `Open 3D Version: 0.16.0`. We are set up, and we can move on to loading the datasets.

### 3.2. Loading datasets

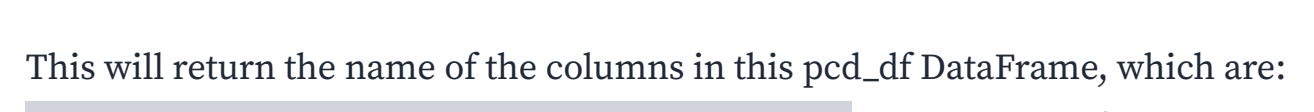
Now we will define the specific paths where our datasets are stored. I like to make it clear and relative to my code file. This way, everything is expressed relatively (the `../` means go to the parent folder) in my current notebook, making navigating the folder easy if I need to code on different machines.

```
data_folder="../DATA/"
pc_dataset="30HZ1_18_sampled.xyz"
mesh_dataset="NL.IMBAG.Pand.0637100000139735.obj"
result_folder="../DATA/RESULTS/"
```

**Point Cloud Dataset.** We can prepare the point cloud by first creating a Pandas `DataFrame` object called `pcd_df`, which will host the point cloud data:

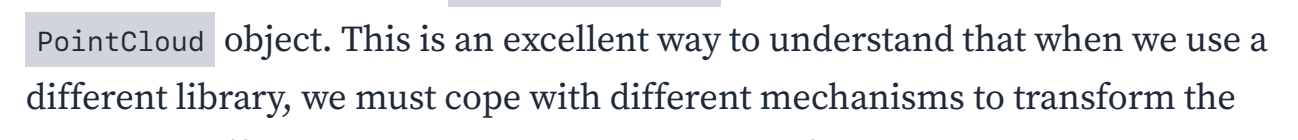
```
pcd_df= pd.read_csv(data_folder+pc_dataset, delimiter=";")
print(pcd_df.columns)
```

This will return the name of the columns in this pcd_df DataFrame, which are: `['X', 'Y', 'Z', 'R', 'G', 'B', 'Classification']`. This is handy for selecting only the explicit "columns" without blurry indexes 😁. And this is precisely what we will do: select only the `['X', 'Y', 'Z']` coordinates to create an Open3D `PointCloud` object. This is an excellent way to understand that when we use a different library, we must cope with different mechanisms to transform the dataset in different Python Objects. Here, we go from a Pandas DataFrame to an Open3D PointCloud, which is necessary to use the Open3D functions implemented in the Open3D library. Let us proceed in four steps, as illustrated below:


1. SELECTING ONLY THE X, Y AND Z COLUMNS FROM THE DATAFRAME
2. TRANSFORMING THE PANDAS OBJECT TO A NUMPY ARRAY OBJECT
3. TRANSFORMING THE NUMPY ARRAY TO AN OPEN 3D VECTOR OBJECT
4. PASSING THIS OPEN3D VECTOR3D OBJECT TO THE POINT CLOUD CONSTRUCTOR

The concept workflow for open3D point cloud creation. © Author

This decomposed mechanism translates into the following:


I1 = pcd_df[['X','Y','Z']]
I2 = np.array(I1)
I3 = o3d.utility.Vector3dVector(I2)
pcd_o3d = o3d.geometry.PointCloud(I3)

The code workflow for open3D point cloud creation. © Author.

But hey, if we want to be a bit more condensed, this transformation can then be done with a single code line, this one:

```
pcd_o3d=o3d.geometry.PointCloud(o3d.utility.Vector3dVector(np.array(pcd_df[[
```

We now have two important variables: `pcd_o3d` and `pcd_df`. We can also give some colors to the Open3D `PointCloud` object:

```
pcd_o3d.colors = o3d.utility.Vector3dVector( np.array( pcd_df[['R','G','B']]
```

We have to be mindful to transform the `R,G,B` values to float values between `[0,1]`, and make sure that we pass a `Vector3dVector` object to the `colors` attribute.

**Mesh Dataset.** Now, we can load the 3D Mesh in a `mesh` variable using the `read_triangle_mesh()` method from `open3d`. We can also paint the mesh with the `paint_uniform_color()` method:

```
mesh=o3d.io.read_triangle_mesh(data_folder+mesh_dataset)
mesh.paint_uniform_color([0.9,0.9,0.9])
```

We now have two Open3D objects: APointCloud with 7 103 848 points and a Triangle Mesh with 674 points and 488 triangles. Let us see what that means, shall we?

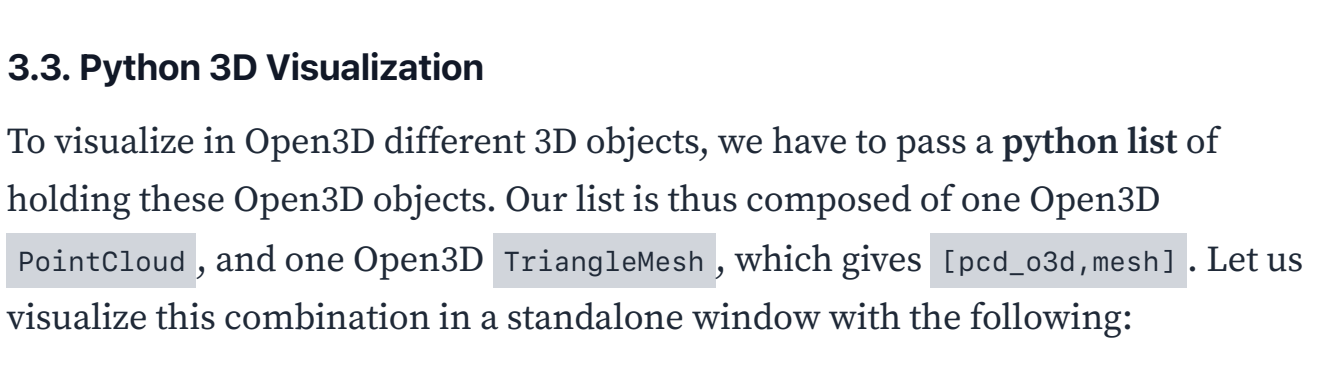
### 3.3. Python 3D Visualization

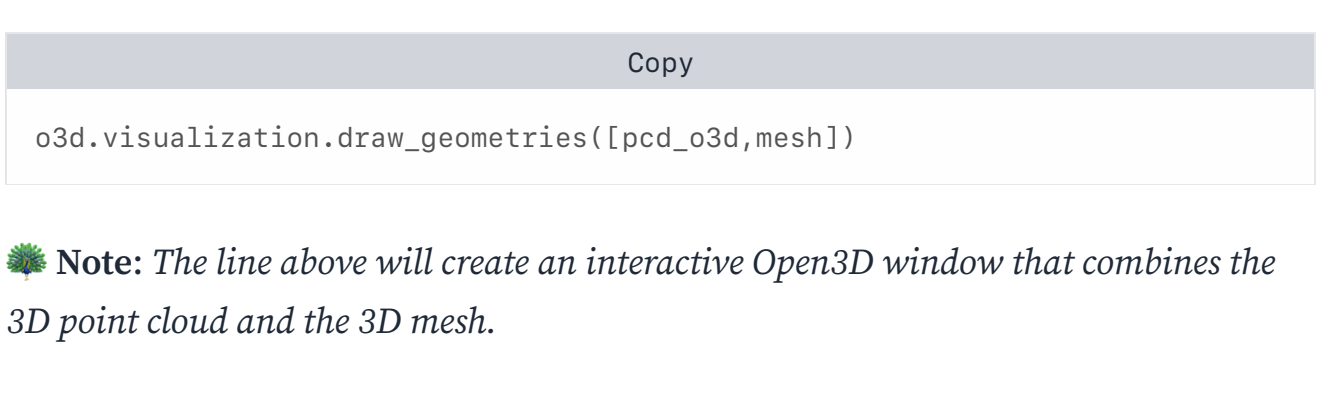
To visualize in Open3D different 3D objects, we have to pass a **python list** of holding these Open3D objects. Our list is thus composed of one Open3D `PointCloud`, and one Open3D `TriangleMesh`, which gives `[pcd_o3d,mesh]`. Let us visualize this combination in a standalone window with the following:

```
o3d.visualization.draw_geometries([pcd_o3d,mesh])
```

🦚 **Note:** *The line above will create an interactive Open3D window that combines the 3D point cloud and the 3D mesh.*

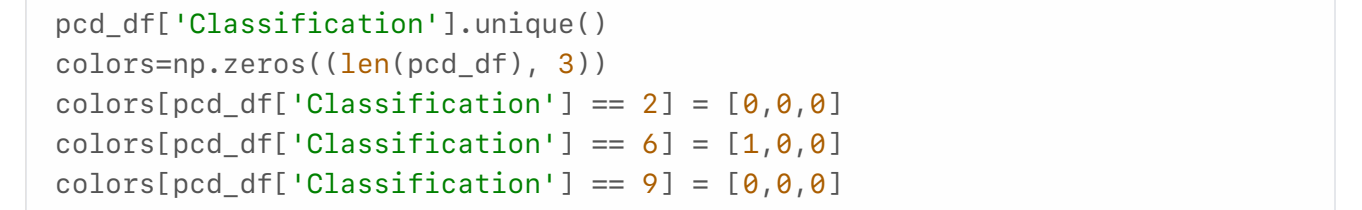
To play with the colors to display, there is one handy trick: using a color variable that we will pass to the colors attribute of the PointCloud Open3D object. This variable should hold R, G, B float values ranging from `0` to `1`.





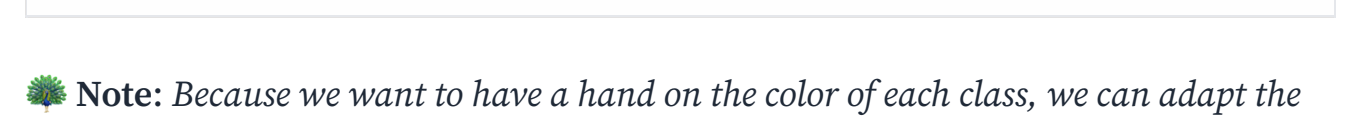
🦚 **Note:** *Do you see the slight difference between both screenshots? On the right side, we can better delineate the borders, and we have a bit more of a depth impression (this is more noticeable interactively). This is because, in the second case, we also use normals for visualization. To do that, you can run the line* `pcd_o3d.estimate_normals()` *and* `mesh.compute_vertex_normals()` *before running the visualization part. Enjoy* 😉

Let us say that we want to visualize the point cloud colored based on the classification attribute. What we thus need to do is to follow this four-stage process:


1. SELECTING THE UNIQUE CLASSIFIED ELEMENTS IN THE POINT CLOUD
2. INITIALIZING AN EMPTY NUMPY ARRAY WITH A SIZE OF (N,3)
3. SELECTING POINTS OF EACH UNIQUE CLASS AND CHANGING THE COLOR VALUE
4. CREATING A 3DVECTOR OBJECT PASSED TO THE COLOR ATTRIBUTE

The concept workflow to use classification as a color for Open3D.

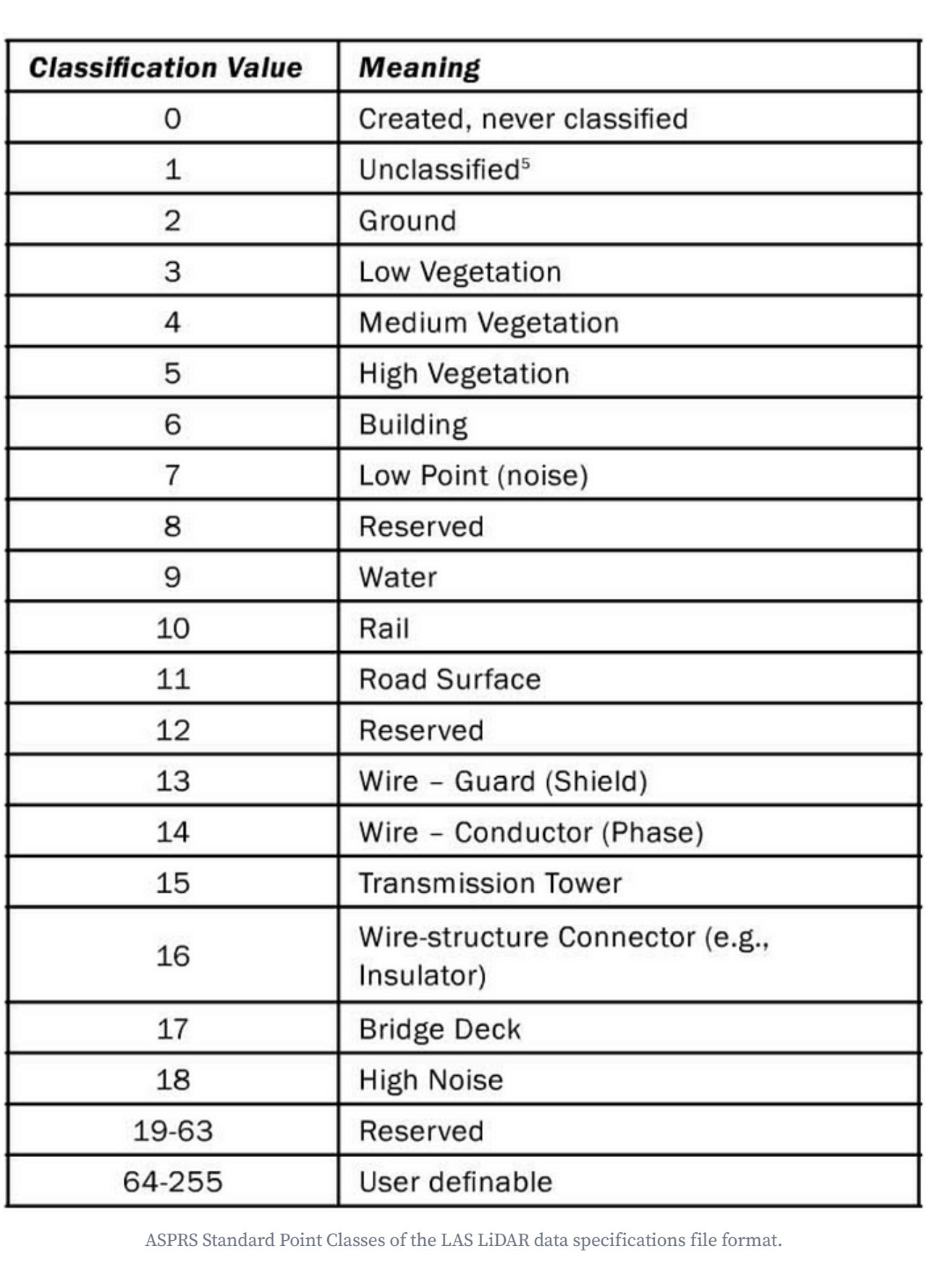
If you carefully notice, this then translates into code such as:


pcd_df['Classification'].unique()
colors=np.zeros((len(pcd_df), 3))
colors[pcd_df['Classification'] == 2] = [0,0,0]
pcd_o3d.colors = o3d.utility.Vector3dVector(colors)

The code workflow to use classification as a color for Open3D.

```
# 1=unclassified, 2=Ground, 6=building, 9=water, 26=rest
pcd_df['Classification'].unique()
colors=np.zeros((len(pcd_df), 3))
colors[pcd_df['Classification'] == 2] = [0,0,0]
colors[pcd_df['Classification'] == 6] = [1,0,0]
colors[pcd_df['Classification'] == 9] = [0,0,0]
colors[pcd_df['Classification'] == 26] = [0,0,0]
pcd_o3d.colors = o3d.utility.Vector3dVector(colors)
```

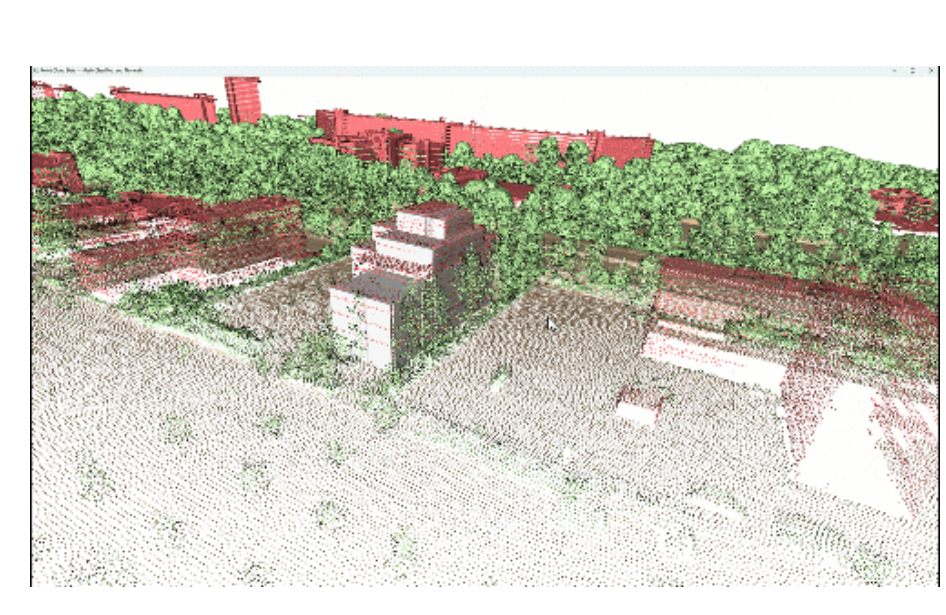
🦚 **Note:** *Because we want to have a hand on the color of each class, we can adapt the code above to the number of unique classes. Then, the correspondence is done based on the* *LAS (1.4) Specifications*.



| ***Classification Value*** | ***Meaning*** |
|---|---|
| 0 | Created, never classified |
| 1 | Unclassified[5] |
| 2 | Ground |
| 3 | Low Vegetation |
| 4 | Medium Vegetation |
| 5 | High Vegetation |
| 6 | Building |
| 7 | Low Point (noise) |
| 8 | Reserved |
| 9 | Water |
| 10 | Rail |
| 11 | Road Surface |
| 12 | Reserved |
| 13 | Wire – Guard (Shield) |
| 14 | Wire – Conductor (Phase) |
| 15 | Transmission Tower |
| 16 | Wire-structure Connector (e.g., Insulator) |
| 17 | Bridge Deck |
| 18 | High Noise |
| 19-63 | Reserved |
| 64-255 | User definable |

ASPRS Standard Point Classes of the LAS LiDAR data specifications file format.

We could visualize the results that would give something like this (with different colors depending on the `R,G,B` values inputted):



Open3D interactive multi-modal data visualization. © Author.

We have variables loaded, we can see both the point cloud and the 3D Mesh, and everything looks like it is working smoothly! Time to define the various 3D City Analytical tasks we want to do!

### 3.4. 3D City Analysis

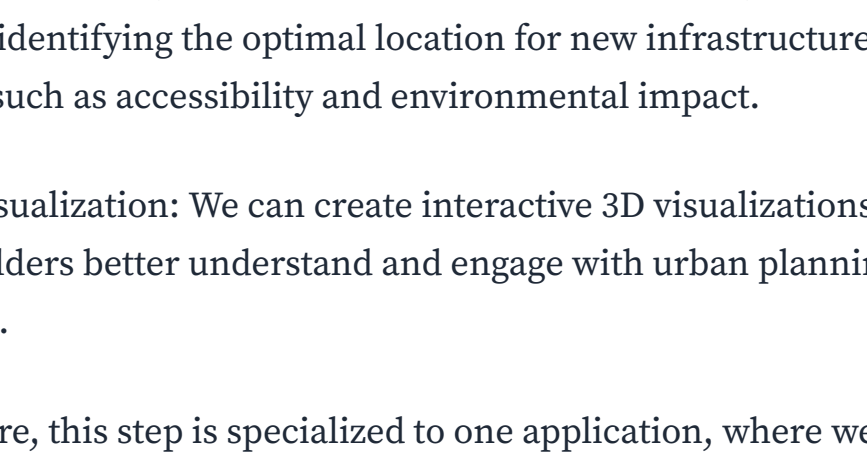
3D city analysis refers to the process of using three-dimensional (3D) models of urban environments to analyze and understand various aspects of the built environment, such as building energy performance, urban morphology, and pedestrian movement. This analysis typically involves using specialized paradigms and conducting analyses to gain insights into urban planning and design. 3D city analysis can be used by urban planners, architects, and engineers to inform decisions about the placement of infrastructure, the design of public spaces, and the mitigation of various environmental impacts. In this tutorial, we will touch on three main aspects of 3D City Analysis:

1. Urban Morphology analysis: We can use Python to analyze the shape and form of buildings in 3D city dataset, which can help inform decisions about urban design and planning.

2. Geospatial Znalysis: We can perform geospatial analysis on the city model, such as identifying the optimal location for new infrastructure projects based on factors such as accessibility and environmental impact.

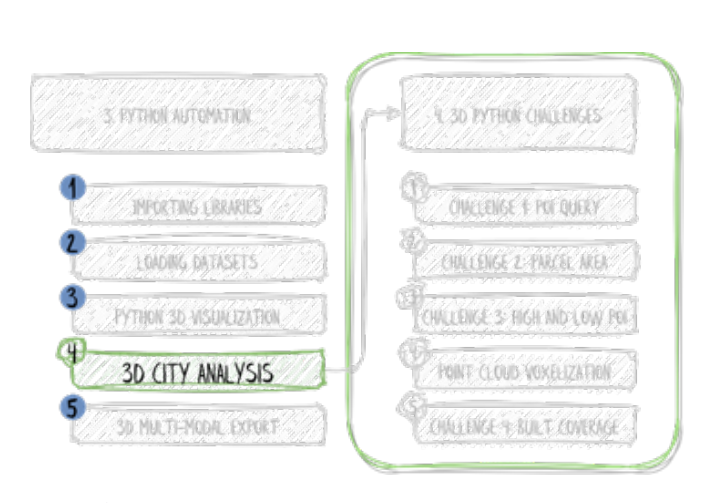
3. 3D Visualization: We can create interactive 3D visualizations, which can help stakeholders better understand and engage with urban planning and design projects.

Therefore, this step is specialized to one application, where we do most of the analysis. We illustrate on 3D City Analysis, which we extend in section 4, through the various Python Challenges, as illustrated below.


3. PYTHON AUTOMATION
1 IMPORTING LIBRARIES
2 LOADING DATASETS
3 PYTHON 3D VISUALIZATION
4 3D CITY ANALYSIS
5 3D MULTI-MODAL EXPORT
4. 3D PYTHON CHALLENGES
CHALLENGE 1: POI QUERY
CHALLENGE 2: PARCEL AREA
CHALLENGE 3: HIGH AND LOW POI
POINT CLOUD VOXELIZATION
CHALLENGE 4: BUILT COVERAGE

3D Analysis in the context of cities, and its decomposition in Step 4: 3D Python Challenges (Section 4).

However, we can follow a common workflow, adapted to each application, which is usually composed of an input, a processing pipeline, and an output for each specific analytical part.


INPUT (E.G NUMPY ARRAY)
| X Y Z |
| X Y Z |
| X Y Z |
PROCESSING PIPELINE (E.G LOW POINT?)
OUTPUT (E.G INTEGER VALUE)
12.543 meters

The conceptual workflow from input to output in the context of point cloud data.

The input can vary, but in most cases, it is a NumPy array that holds the spatial information. The output can be an integer, a list, another NumPy array, … Anything you need in return. 😁

🦚 **Note:** *This specific stage is dense, and we will leave some space in our notebook for the different analyses linked to the challenges we will see in Step 4. For now, let us move to define some way to export our data.*

### 3.5. 3D Multi-Modal Export

After our 3D pipeline is fully functional, we can save the results to one or more files to work them outside Python. For this, we will use two valuable possibilities.

1. **Numpy (Recommended for complex outputs):** We can export with Numpy with the following line of code: `np.savetxt(result_folder+pc_dataset.split(".")[0]+"_selection.xyz", np.asarray(o3d_parcel_corners),delimiter=';', fmt='%1.9f')`
2. **Open3D (Recommended when needing only spatial attributes):** We can Export with Open3D with the following line of code: `o3d.io.write_point_cloud(result_folder+pc_dataset.split(".")[0]+"_result_filtered_o3d.ply", pcd_selection, write_ascii=False, compressed=False, print_progress=False)`

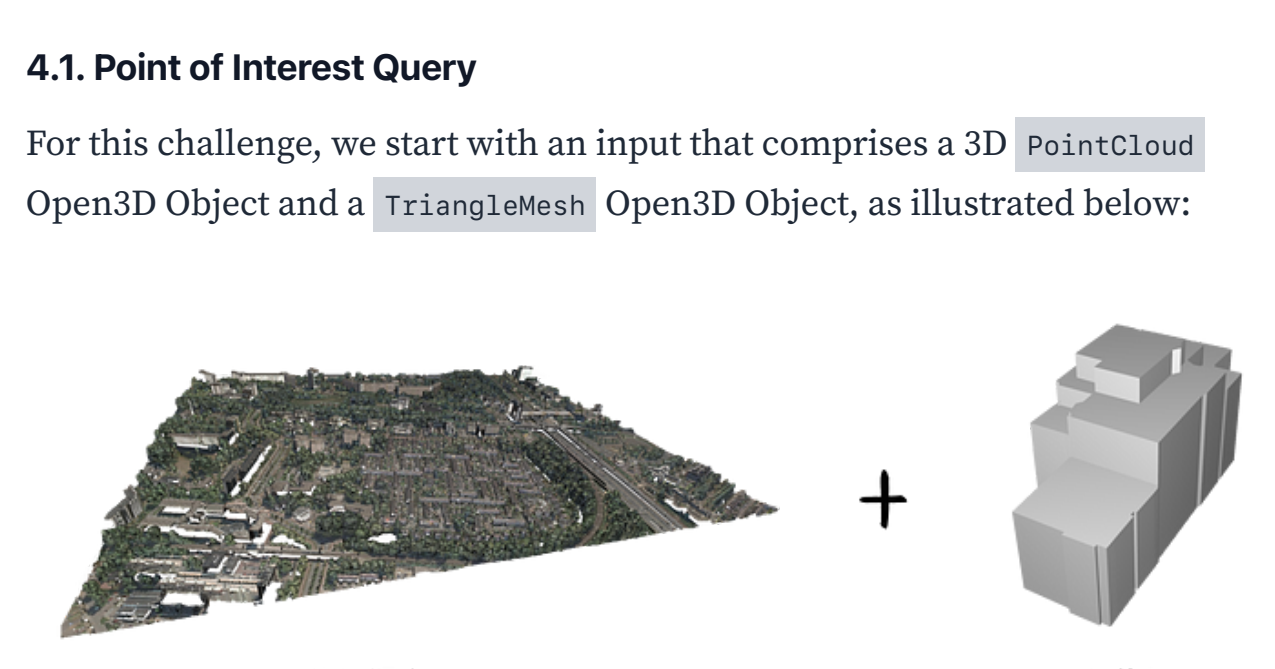
🦚 **Note**: *the* `.split(".")` *allows to split the* `pc_dataset` *string object into a list of two strings, before and after the* `.`, *an then we keep only the first element with* `[0]`. *The NumPy exports a variable* `o3d_parcel_corners` *stage wit a delimiter* `;` *in a* `.xyz` *ASCII file. The Open3D will write a* `.ply` *file from the open3d object* `pcd_selection`.

Wow, well done! The Python Automation bulk structure is up and running! Congratulation! We have the libraries imported, the datasets are stored in different explicit variables, and we ensure we can deal with various visualization without leaving the comfort of Python. The Export step is set up; all that is left is to target the initial question we had in Step 4:

> How dense is the built area of the neighborhood around our house? Can the house be subject to flooding? Am I respecting the built ratio for the parcel I own?

## Step 4. 3D Python Challenges

To answer the questions above, we will find a solution to four challenges and voxelization steps, as illustrated below.


1. ENVIRONMENT SET-UP
SOFTWARE STACK
PYTHON IDE
PYTHON LIBRARIES
2. 3D DATA PREPARATION
DOWNLOADING DATASETS
3D DATA VIZUALISATION
3D DATA FILTERING
3D DATA EXPORT
3. PYTHON AUTOMATION
IMPORTING LIBRARIES
LOADING DATASETS
PYTHON 3D VISUALIZATION
3D CITY ANALYSIS
3D MULTI-MODAL EXPORT
4. 3D PYTHON CHALLENGES
1 CHALLENGE 1: POI QUERY
2 CHALLENGE 2: PARCEL AREA
3 CHALLENGE 3: HIGH AND LOW POI
4 POINT CLOUD VOXELIZATION
5 CHALLENGE 4: BUILT COVERAGE

Step 4: 3D Python Challenges. We investigate Point of Interest queries, Manual Boundary selection, High point extraction, voxelization and built coverage extraction.

Challenge 1 will permit cropping out the study zone to the desired neighborhood. Challenge 2 will permit to extract a built ratio for the owned parcel. Challenge 3 guides the flooding analysis. And Challenge 4, with a voxelization beforehand, will allow getting the built coverage of the area of interest. Let us get started.

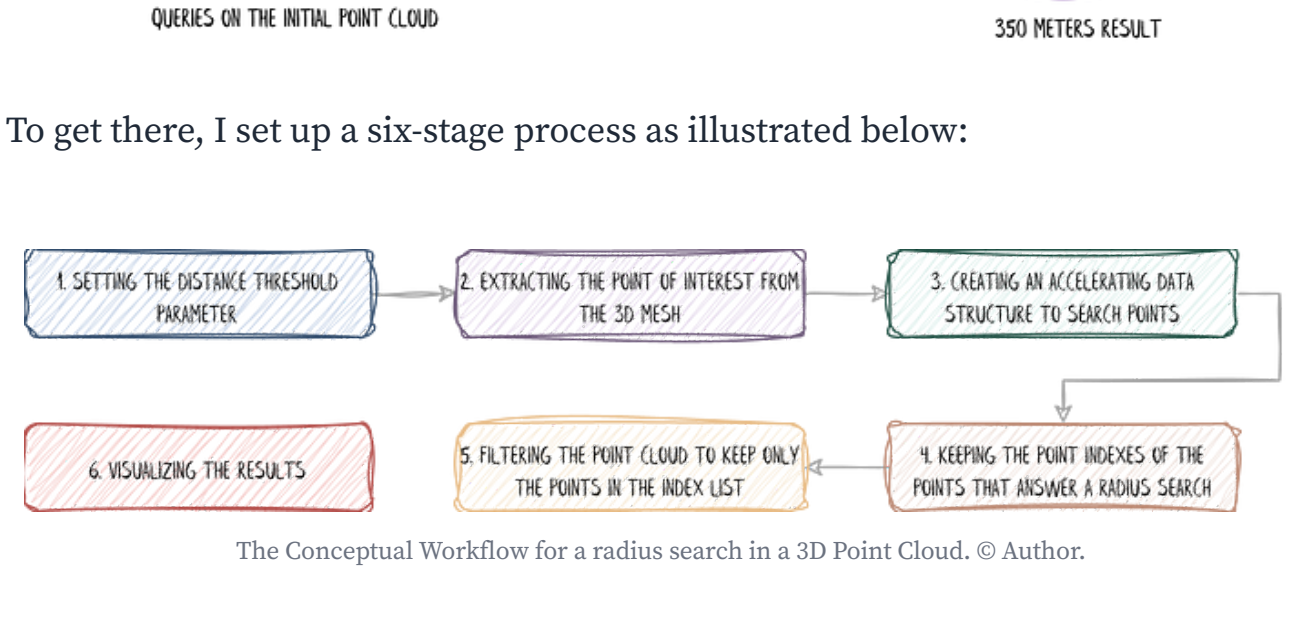
### 4.1. Point of Interest Query

For this challenge, we start with an input that comprises a 3D `PointCloud` Open3D Object and a `TriangleMesh` Open3D Object, as illustrated below:

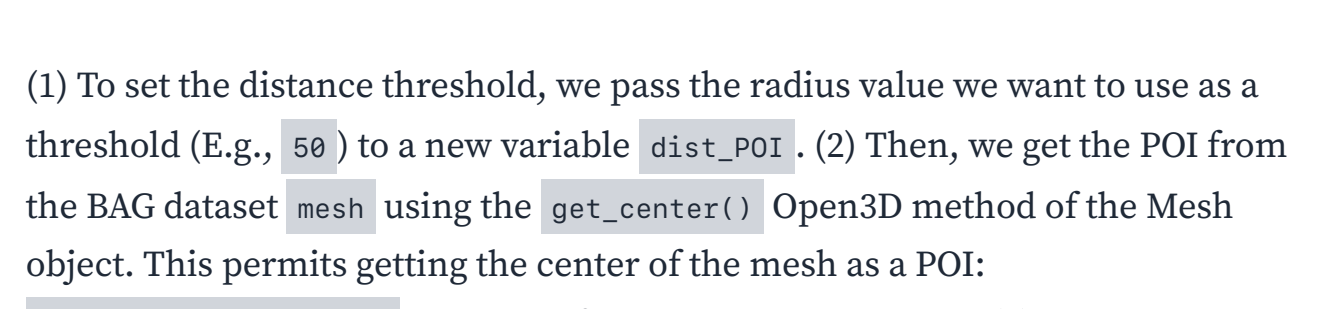
3D POINT CLOUD
+
3D MESH

A 3D Point Cloud with a 3D mesh object. © F. Poux

The objective of this challenge is to keep only the data points that fall within a certain distance from your point of interest, the building house. Our inputs are the point cloud and the mesh, and our output is the filtered point cloud which answers the distance to the POI criterion as shown below:


QUERIES ON THE INITIAL POINT CLOUD
50 METERS RESULT
350 METERS RESULT

To get there, I set up a six-stage process as illustrated below:


1. SETTING THE DISTANCE THRESHOLD PARAMETER
2. EXTRACTING THE POINT OF INTEREST FROM THE 3D MESH
3. CREATING AN ACCELERATING DATA STRUCTURE TO SEARCH POINTS
4. KEEPING THE POINT INDEXES OF THE POINTS THAT ANSWER A RADIUS SEARCH
5. FILTERING THE POINT CLOUD TO KEEP ONLY THE POINTS IN THE INDEX LIST
6. VISUALIZING THE RESULTS

The Conceptual Workflow for a radius search in a 3D Point Cloud. © Author.

🎓 **Learning Note:** *The goal is to try to sort things out on your own and check back if you have troubles. Whenever you are ready, you can read the solution below.* 👇

(1) To set the distance threshold, we pass the radius value we want to use as a threshold (E.g., `50`) to a new variable `dist_POI`. (2) Then, we get the POI from the BAG dataset `mesh` using the `get_center()` Open3D method of the Mesh object. This permits getting the center of the mesh as a POI: `POI=mesh.get_center()`. We can, after that, create a KD-tree (3), a data structure used to organize points in space. KD trees are helpful for point cloud processing because they allow for fast nearest-neighbor searches and range queries. These are good news then because this is what we are doing 😁. Indeed, using a KD-tree makes it possible to find the points closest to a given point quickly, or all the points within a certain distance (our POI), without having to search through all the points in the point cloud.

```
# Creating a KD-Tree Data Structure
pcd_tree = o3d.geometry.KDTreeFlann(pcd_o3d)
```

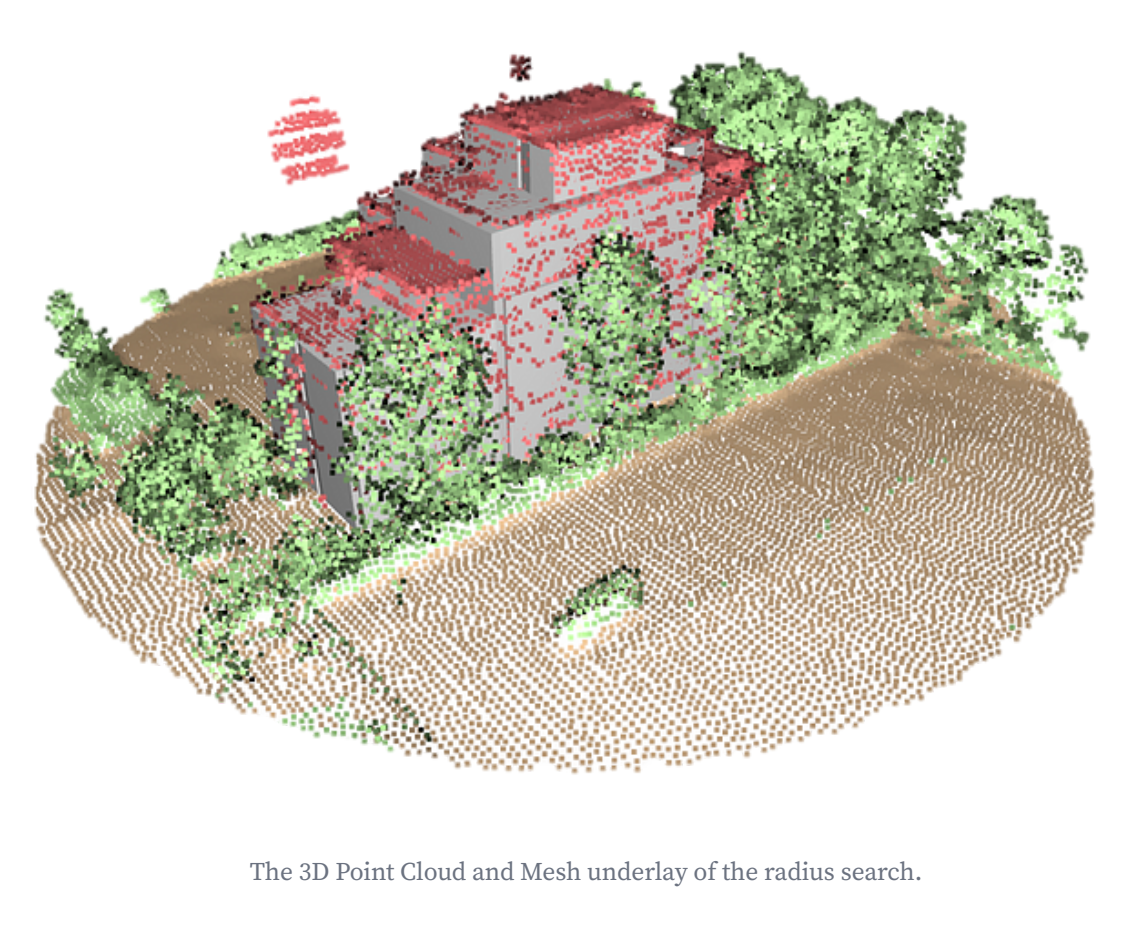
🦚 **Note:** *The KD-Tree works by recursively partitioning the space into smaller regions, dividing it along the median of one of the dimensions at each level (*`X` *is a dimension,* `Y` *another,* `Z` *another* 😉*). This results in a "tree" structure where each node represents a space partition, and each leaf node represents a single point.*

From there, we can then use the `search_radius_vector_3d()` method of Open3D and select the points from the output index: to finally visualize our result (`4` + `5` + `6`):

```
# Selecting points by distance from POI (your house) using the KD-Tree
[k, idx, _] = pcd_tree.search_radius_vector_3d(POI, dist_POI)
pcd_selection=pcd_o3d.select_by_index(idx)
```

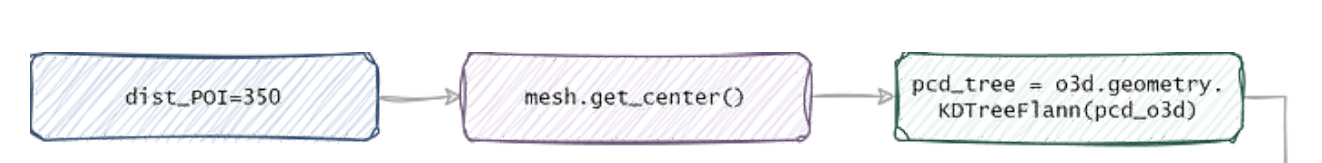
We can finally visualize our results (6):

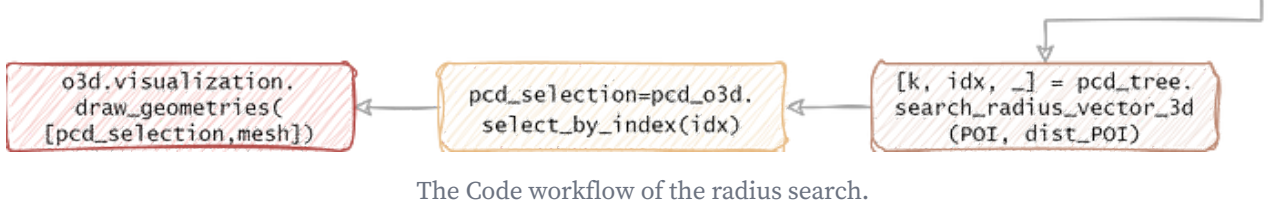
```
o3d.visualization.draw_geometries([pcd_selection,mesh])
```



The 3D Point Cloud and Mesh underlay of the radius search.

Which amount, all in all, to the following code pipeline:


dist_POI=350
mesh.get_center()
pcd_tree = o3d.geometry.KDTreeFlann(pcd_o3d)


o3d.visualization.
draw_geometries(
[pcd_selection,mesh])
pcd_selection=pcd_o3d.
select_by_index(idx)
[k, idx, _] = pcd_tree.
search_radius_vector_3d
(POI, dist_POI)

The Code workflow of the radius search.

Very good! Now that you have a working solution let us extract the parcel area from that selection.

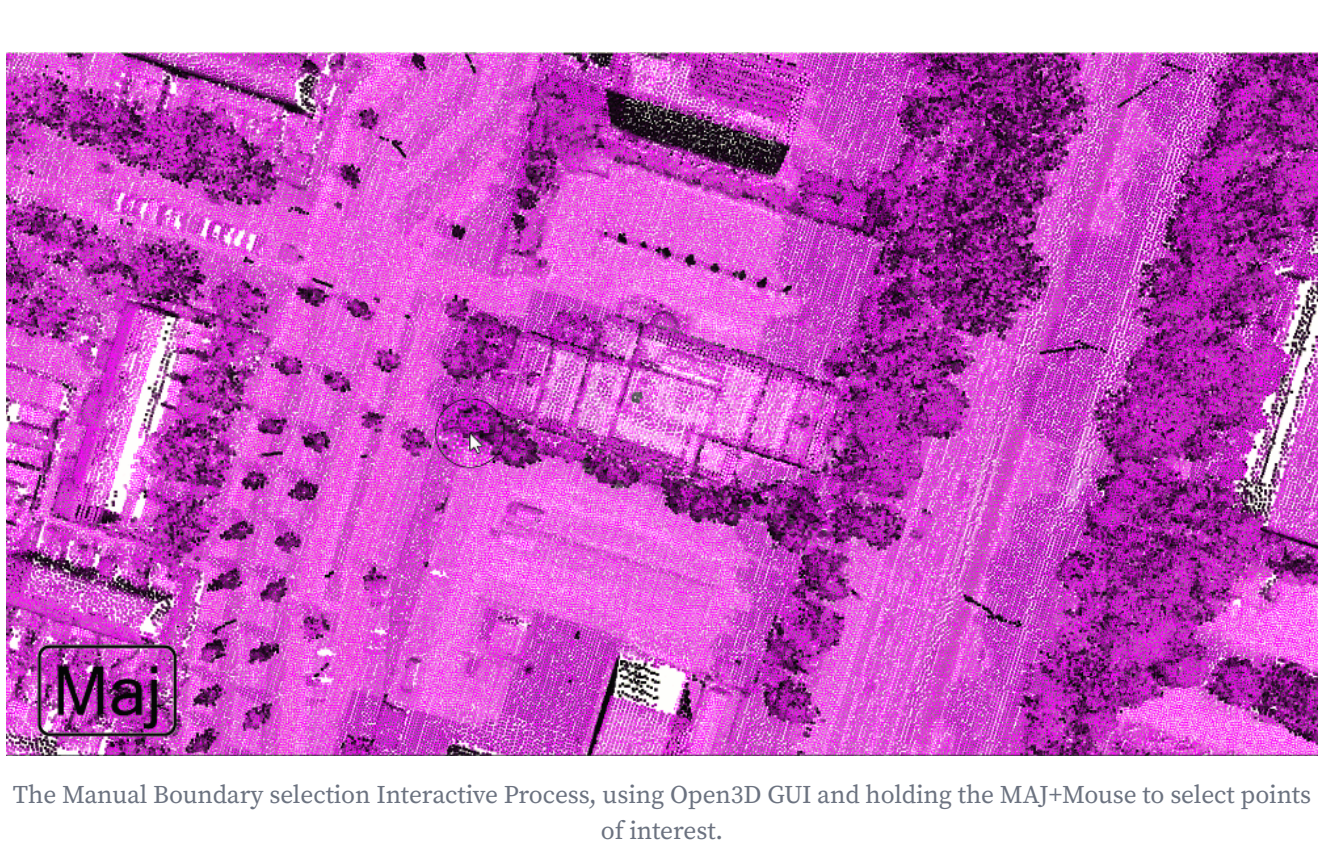
### 4.2. Manual Boundaries Selection

To extract the unofficial boundaries, we have to move onto some semi-automated and interactive approach. The good news is that we can do that directly within Python with Open3D.

The first thing to do is to create an interactive Open3D window with the following `draw_geometries_with_vertex_selection()` method:

```
o3d.visualization.draw_geometries_with_vertex_selection([pcd_selection])
```

You can then follow the animated part below, which allows selecting the points that define the corners of your parcel.



The Manual Boundary selection Interactive Process, using Open3D GUI and holding the MAJ+Mouse to select points of interest.

The results will then appear in the results under your cell in your notebook (or your REPL) upon closing the window.

🦚 **Note:** *It may be easier at this step to use the* `R`*,* `G`*,* `B` *coloring. You would thus have to move back to change this before the selection if you want to work with the correct indexes.* 😉 *If you want to extract the cadastral boundary, this is feasible, by importing the official 2D vector shape and cutting based on this data constraint.*

From your REPL, you can copy and paste the different indexes (E.g., `34335`, `979`, `21544`, `19666`, `5924`, `21816`, `38008`) of the selected points into the `select_by_index()` selection method to define a `o3d_parcel_corners` variable:

```
o3d_parcel_corners=pcd_selection.select_by_index([34335 ,979 ,21544 ,19666 ,
```

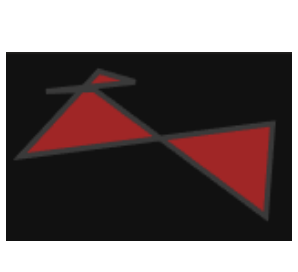
We still have to prepare the corners further because we want to avoid considering the `Z` value. Therefore, we will filter out the coordinates to drop the Z value, but beware: doing this means that we consider that we are in a flat area (which is the case in the Netherlands 😉).

```
o3d_parcel_corners=np.array(o3d_parcel_corners.points)[:,:2]
```

From there, it is time to compute the area of the parcel with the `Shapely` library! For this, we can directly use `Polygon` function first to create a polygon out of the set of provided corners:

```
Polygon(o3d_parcel_corners)
```

Which outputs the following:



wrong geometry

This looks somewhat wrong, doesn't it? If you do not believe me, we can compute the area to check:

```
pgon = Polygon(o3d_parcel_corners)
print(f"This is the obtained parcel area: {pgon.area} m²")
```

Which outputs `43.583 m²`. That sounds weird for a big building, doesn't it? Ha, a simple problem becomes a bit more complicated! Indeed, the problem here is that computing the area needs a polygon that is constituted in a way we obtain a closed shape. This is not always the case because the order corners have been added to the data structure. Thus, the problem becomes sorting out the coordinates to avoid any edges intersection.

One way of dealing with the problem is to perceive all coordinates from the perspective of the center point. We can then compute the angle between each corner in the list and our center point. We do this to have an idea of how wide the individual angles are and therefore provide us with the means to sort out the coordinates based on their values. Translating this into code allows defining a sorting function as follows:

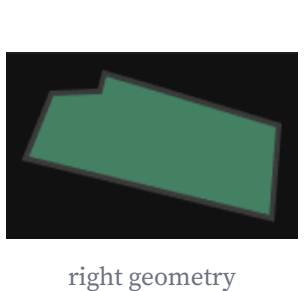
```
def sort_coordinates(XY):
    cx, cy = XY.mean(0)
    x, y = XY.T
    angles = np.arctan2(x-cx, y-cy)
    indices = np.argsort(-angles)
    return XY[indices]
```

🦚 **Note:** *I use the NumPy* `arctan2()` *method to perform angle estimation for every coordinate. This will return the array of angles in radians. All that's left is to sort out the angles in ascending order to receive a list of indices in the correct order. The list can then fix the original list indices with the* `argsort()` *method. At this step, it will not work with U-shape buildings.*

From there, we can apply our sorting function to the corners, create a new sorted variable, compute the polygon and the associated area:

```
np_sorted_2D_corners=sort_coordinates(o3d_parcel_corners)
pgon = Polygon(np_sorted_2D_corners)
Polygon(np_sorted_2D_corners)
print(f"This is the parcel area: {pgon.area} m²")
```

This returns the following polygon with an area of `2 247.14 m²`, which is much more plausible. 😉



right geometry

Well done again. We now have a good idea of the area of our parcel. Now, let us find the high and low points in the zone

### 4.3. Find high and low points

To get the low and high points of the area, one specific way would be to use the `get_max_bound()` and `get_min_bound()` methods of Open3D:

```
pcd_selection.get_max_bound()
pcd_selection.get_min_bound()
```

However, doing this will not hint which point is the highest and which is the lowest. What we need are the point indexes to retrieve the coordinates after that. For this, I propose that we code it this way:

1. We create a NumPy array object, `np_pcd_selection`, from the Open3D PointCloud object that holds only the `X`, `Y` and `Z` coordinates.
2. We gather the indexes of the min and max values over the `Z` dimension with the `argmax()` method and store the results in the variables `lowest_point_index` and `highest_point_index`.

```
np_pcd_selection=np.array(pcd_selection.points)
lowest_point_index=np.argmin(np_pcd_selection[:,2])
highest_point_index=np.argmax(np_pcd_selection[:,2])
```

Let us check the results by selecting using the indexes, creating `TriangleMesh` Spheres, translating them to the position of the points, and then visualizing with Open3D:

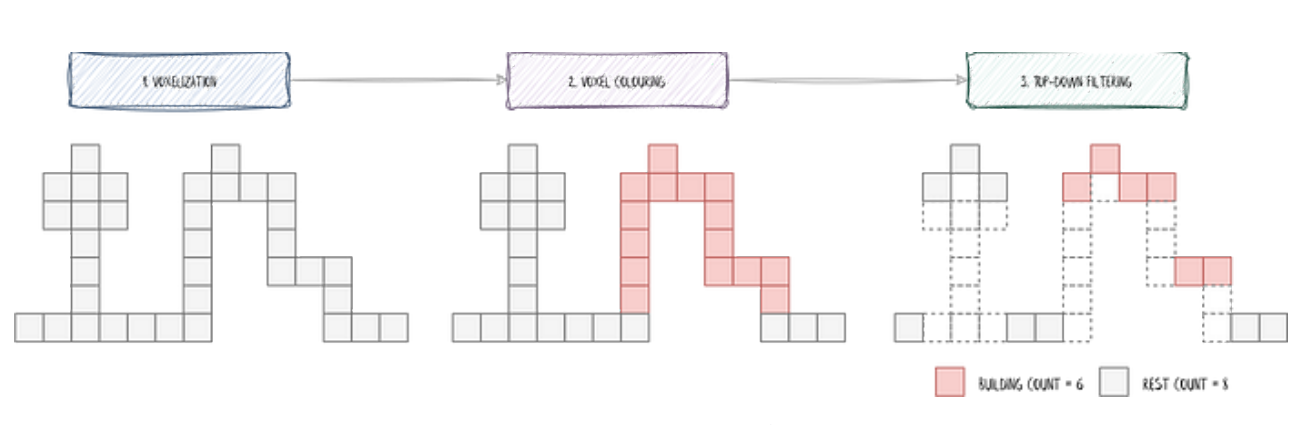
```
# We create 3D Spheres to add them to our visual scene
lp=o3d.geometry.TriangleMesh.create_sphere()
hp=o3d.geometry.TriangleMesh.create_sphere()

# We translate the 3D Spheres to the correct position
lp.translate(np_pcd_selection[lowest_point_index])
hp.translate(np_pcd_selection[highest_point_index]))

# We compute some normals and give color to each 3D Sphere
lp.compute_vertex_normals()
lp.paint_uniform_color([0.8,0.1,0.1])
hp.compute_vertex_normals()
hp.paint_uniform_color([0.1,0.1,0.8])

# We generate the scene
o3d.visualization.draw_geometries([pcd_selection,lp,hp])
```

As shown below, we now have our 3D scene with the high and low points.

Let us study the building coverage in an extended vicinity of 350 meters. For this, we will re-execute the code for some parts, as explained below.

### 4.4. Point Cloud Voxelization

We want to extract the built coverage. For this, we will take an intuitive approach by first transforming the point cloud modality to a filled analog of the 2D pixels: 3D voxels. This will allow us to pursue the following methodology:


1. VOXELIZATION
2. VOXEL COLOURING
3. TOP-DOWN FILTERING
BUILDING COUNT = 6
REST COUNT = 5

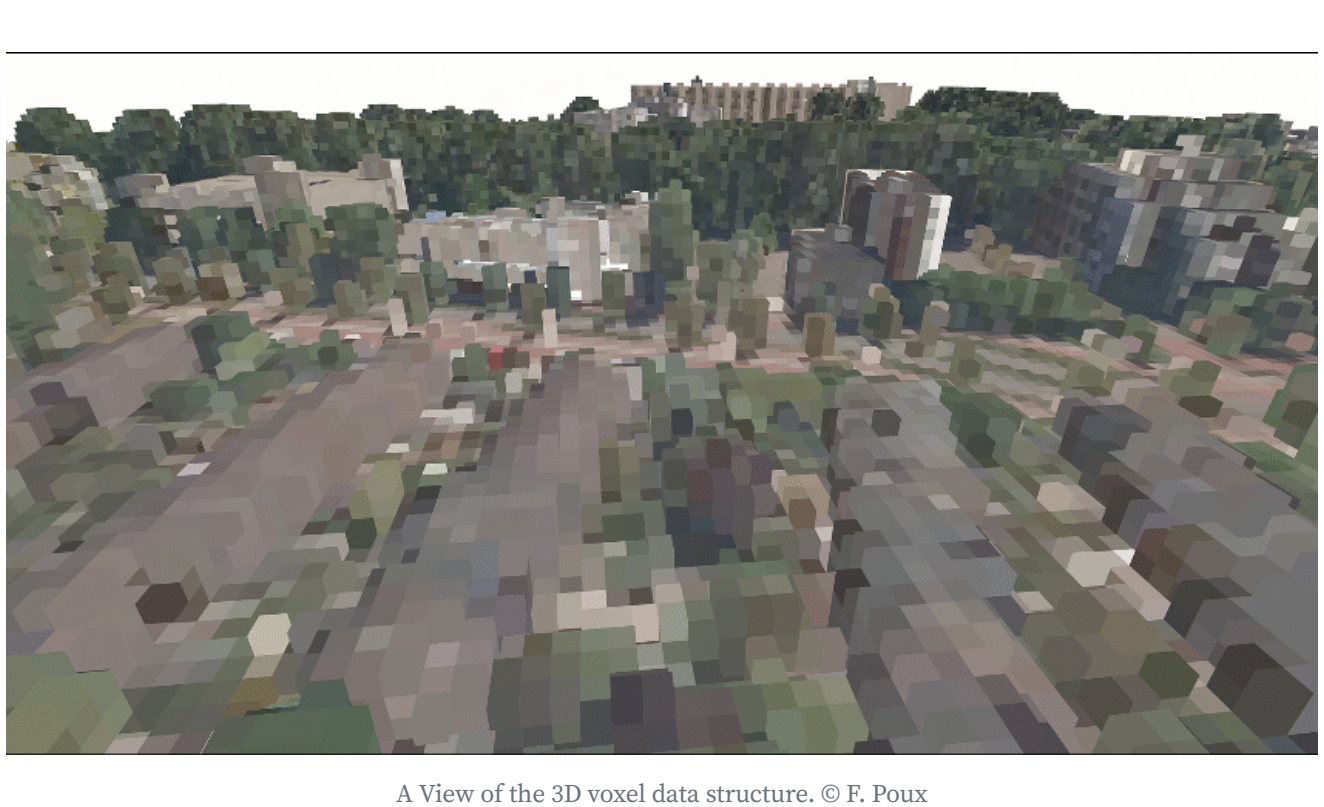
Extracting high points by constructing a 3D voxel data structure. We first voxelize the point cloud, then we color each voxel from a binary perspective, and then we filter by top-down indexes. © F. Poux

Basically, we will (1) generate voxels where points exist, then (2) we will color the voxels based on the classification of the points they hold, and finally, we will filter the voxels to keep and count only the highest voxel per X,Y coordinate. This way, we can avoid counting elements that would bias the built coverage.

The first step is to create a voxel grid. This is where Open3D shines: with these simple lines of code, we can fit a voxel grid to the point cloud, where each voxel is a cube of 20 cm, then visualize the results:

```
voxel_grid = o3d.geometry.VoxelGrid.create_from_point_cloud(pcd_selection, v
o3d.visualization.draw_geometries([voxel_grid])
```

---



A View of the 3D voxel data structure. © F. Poux

🦚 **Note:** *The result you see has been through a change in the coloring of the voxels beforehand. This change means, with Open3D, that we have to change the color of the voxel's points. Then, the VoxelGrid method will average the colors and retain the result as a color for the voxel.*

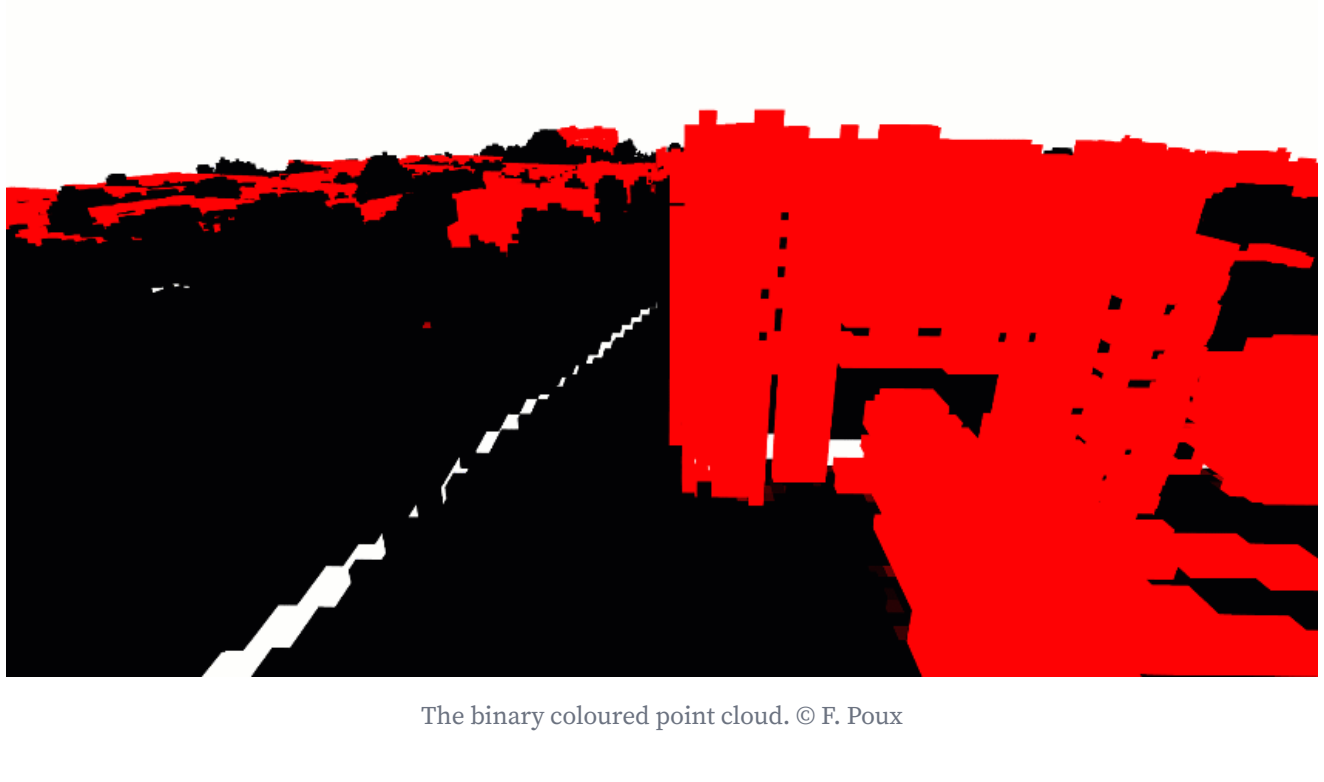
Now that we know how to generate 3D voxels from point clouds let us play on their color scheme to bypass the coloring averaging problem. For this, a simple way would be to handle the color in a binary way. Either it is black ([0,0,0]) or not (all the rest). This means that we can initialize the colors variable to 0, and then select all the points which are classified as a building, and give them another color, E.g., Red ([1,0,0]):

```
colors=np.zeros((len(pcd_df), 3))
colors[pcd_df['Classification'] == 6] = [1, 0, 0]
pcd_o3d.colors = o3d.utility.Vector3dVector(colors)
```

Very nice! Now, because we played with the original point cloud, we will need to redefine the POI and the selection to our choosing. For the sake of efficiency, you will find the code block that you can run to update your voxel rendering to the new color scheme:

```
# Defining the POI and the center of study
dist_POI=50
POI=mesh.get_center()

# Querying all the points that fall within based on a KD-Tree
pcd_tree = o3d.geometry.KDTreeFlann(pcd_o3d)
[k, idx, _] = pcd_tree.search_radius_vector_3d(POI, dist_POI)
pcd_selection=pcd_o3d.select_by_index(idx)

#Computing the voxel grid and visualizing the results
voxel_grid = o3d.geometry.VoxelGrid.create_from_point_cloud(pcd_selection, v
o3d.visualization.draw_geometries([voxel_grid])
```

---



The binary coloured point cloud. © F. Poux

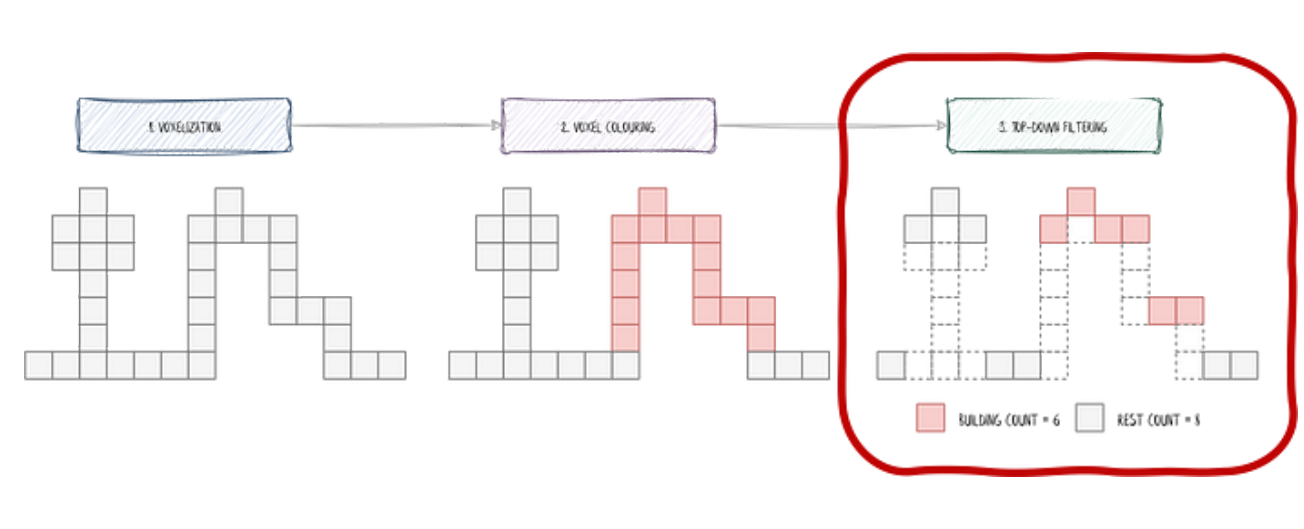
Awesome! now, we need to actually get the discrete integer indexes of each voxel in a list, as well as the colors and the bounds. This will permit us to loop over each voxel and its color later to check if a voxel is the highest in a "voxel column." To do this, an efficient way is to do a list comprehension to vectorize our computation and avoid unnecessary loops:

```
idx_voxels=[v.grid_index for v in voxel_grid.get_voxels()]
color_voxels=[v.color for v in voxel_grid.get_voxels()]
bounds_voxels=[np.min(idx_voxels, axis=0),np.max(idx_voxels, axis=0)]
```

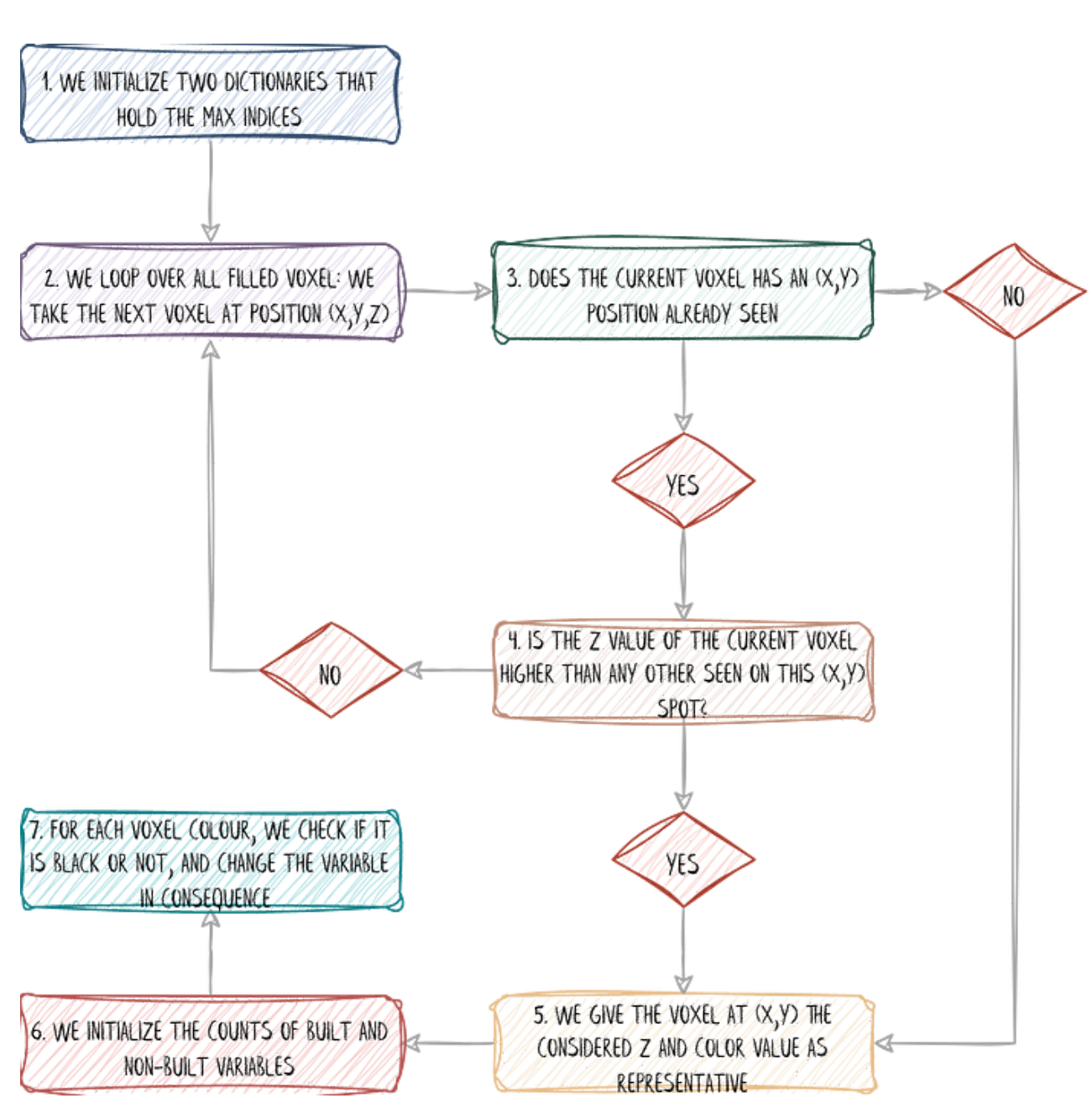
This is awesome! On a selection of 50 meters, we have a grid of 49 x 49x 16 voxels, which amounts to 38 416 filled voxels. Time to extract the built coverage!

### 4.5. Extract the Built Coverage

Now that we have a voxelized point cloud with binary colors, we focus on the third stage of the illustration below:


1. VOXELIZATION
2. VOXEL COLOURING
3. TOP-DOWN FILTERING
BUILDING COUNT = 6
REST COUNT = 5

As we see, selecting only the top voxels is a bit more complex than it seems! 😁 But lucky you, I designed a simple yet robust pipeline that will do just that, below.


1. WE INITIALIZE TWO DICTIONARIES THAT HOLD THE MAX INDICES
2. WE LOOP OVER ALL FILLED VOXEL: WE TAKE THE NEXT VOXEL AT POSITION (X,Y,Z)
3. DOES THE CURRENT VOXEL HAS AN (X,Y) POSITION ALREADY SEEN
NO
YES
4. IS THE Z VALUE OF THE CURRENT VOXEL HIGHER THAN ANY OTHER SEEN ON THIS (X,Y) SPOT?
NO
YES
7. FOR EACH VOXEL COLOUR, WE CHECK IF IT IS BLACK OR NOT, AND CHANGE THE VARIABLE IN CONSEQUENCE
6. WE INITIALIZE THE COUNTS OF BUILT AND NON-BUILT VARIABLES
5. WE GIVE THE VOXEL AT (X,Y) THE CONSIDERED Z AND COLOR VALUE AS REPRESENTATIVE

The Voxel Selection Workflow to extract the built coverage in seven sub-step. © F. Poux.

Let us go through it step by step.

(1) First, we initialize two dictionaries that hold the max indices:

```
max_voxel={}
max_color={}
```

(2 to 5) We loop over all filled voxels to check if they are the highest in a voxel column or to be dropped, which gives the following for loop :

```
for idx, v in enumerate(idx_voxels):
    if (v[0],v[1]) in max_voxel.keys():
        if v[2]>max_voxel[(v[0],v[1])]:
            max_voxel[(v[0],v[1])]=v[2]
            max_color[(v[0],v[1])]=color_voxels[idx]
    else:
        max_voxel[(v[0],v[1])]=v[2]
        max_color[(v[0],v[1])]=color_voxels[idx]
```

🦚 **Note:** *The first thing to notice is the* `enumerate()` *method in the loop definition. This permits looping over each value of the variable* `idx_voxels` *while keeping track of the index of the list. Handy! The second thing is that we are using the "tuples" data type as* `(X, Y)` *, which gives the integer position of our voxel. This permits us to ensure that we are continuously checking on the identical* `X`*,* `Y` *grid position. Finally, the "* `if` *" statement permits testing the expressed condition and will execute if the condition returns* `True`*. If it does not, it will execute the* `else` *statement (in the case there is one) or pass and exit the condition check.*

(6) We initialize the counts of voxels tagged as built or non-built, and we check if the color of the top voxel retained is black or not, in which case we update the voxel count of the respective category:

```
count_building_coverage,count_non_building=0,0
for col in list(max_color.values()):
    if np.all(col==0):
        count_non_building+=1
    else:
        count_building_coverage+=1
```

🦚 **Note:** `np.all()` *is also a boolean check and will return* `True` *only if all values within are* `True`*. In our case, the black color is* `[0,0,0]`*, which will thus return* `True` *because all* `R`*,* `G`*, and* `B` *are set to* `0` *in this case. If one of them is not zero, that means that not all are* `0` *and the* `np.all()` *will return* `False`*, which will trigger the* `else` *statement. Easy?* 😉

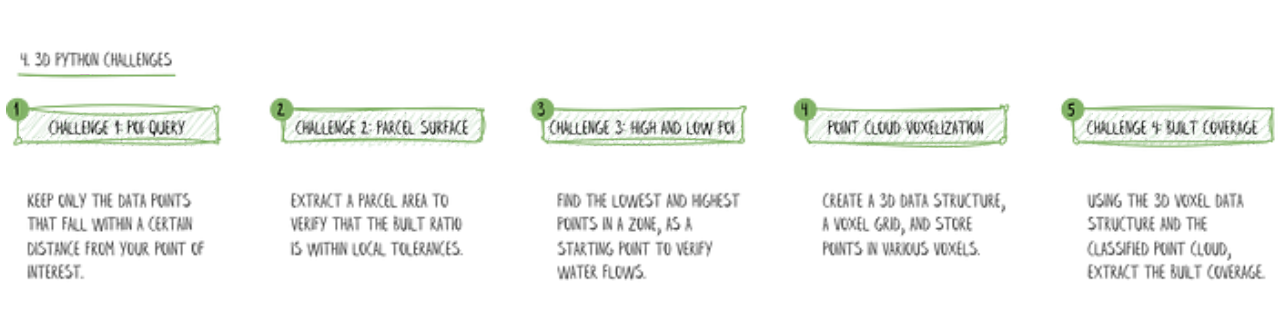
(7) We can extract the area covered for each type (built and non-built) without forgetting to multiply the count variable by the actual 2D voxel area (4 m² in our case) and get the ratio:

```
print(f"Coverage of Buildings: {count_building_coverage*4} m²")
print(f"Coverage of the Rest: {count_non_building*4} m²")
print(f"Built Ratio: {(count_building_coverage*4)/(count_building_coverage*2-
```

This gives us, for 50 meters, a coverage of 17.3%, which amount to 1352 m² of built area and 6456 m² belonging to the rest. (19.2%, 73416 m² and 308280 m² for the 350 meters query).

So, we have a nice balance of building occupation on the selected point of interest compared to the rest!

Hopefully, putting our 3D Python Workflow to the test was not too nerve-wracking! Feel free to return regularly to the code and workflow snippets to master the hidden code tricks that permit (1) to answer with brio the challenges and (2) optimize the efficiency of our implementation.

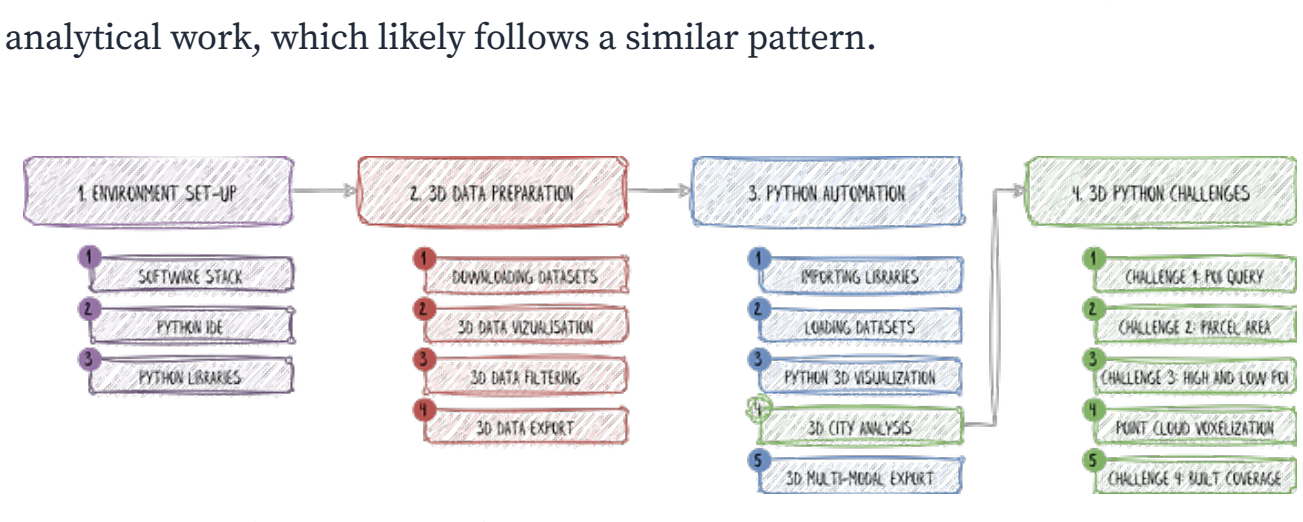

4. 3D PYTHON CHALLENGES
1
CHALLENGE 1: POI QUERY
KEEP ONLY THE DATA POINTS THAT FALL WITHIN A CERTAIN DISTANCE FROM YOUR POINT OF INTEREST.
2
CHALLENGE 2: PARCEL SURFACE
EXTRACT A PARCEL AREA TO VERIFY THAT THE BUILT RATIO IS WITHIN LOCAL TOLERANCES.
3
CHALLENGE 3: HIGH AND LOW POI
FIND THE LOWEST AND HIGHEST POINTS IN A ZONE, AS A STARTING POINT TO VERIFY WATER FLOWS.
4
POINT CLOUD VOXELIZATION
CREATE A 3D DATA STRUCTURE, A VOXEL GRID, AND STORE POINTS IN VARIOUS VOXELS.
5
CHALLENGE 4: BUILT COVERAGE
USING THE 3D VOXEL DATA STRUCTURE AND THE CLASSIFIED POINT CLOUD, EXTRACT THE BUILT COVERAGE.

Yet, some room remains left for tweaking, such as using the classification information combined with the high-low POI to isolate the ground point for water flows or using the parcel surface selection as a filtering technique for built coverage extraction.

- 💻 Direct access to the hands-on code: Google Colab
- 🪄 Direct access to Point Cloud Processing: Github
- 🧙 Direct access to Point Cloud Courses: 3D Academy

## 🔮 Conclusion

Congratulations! This was an actual giant first leap in the world of 3D Python Workflows using LiDAR for City Modelling! The pipeline that we constructed below is very generic and can be considered a point of reference for your future analytical work, which likely follows a similar pattern.


1. ENVIRONMENT SET-UP
1 SOFTWARE STACK
2 PYTHON IDE
3 PYTHON LIBRARIES
2. 3D DATA PREPARATION
1 DOWNLOADING DATASETS
2 3D DATA VIZUALISATION
3 3D DATA FILTERING
4 3D DATA EXPORT
3. PYTHON AUTOMATION
1 IMPORTING LIBRARIES
2 LOADING DATASETS
3 PYTHON 3D VISUALIZATION
4 3D CITY ANALYSIS
5 3D MULTI-MODAL EXPORT
4. 3D PYTHON CHALLENGES
1 CHALLENGE 1: POI QUERY
2 CHALLENGE 2: PARCEL AREA
3 CHALLENGE 3: HIGH AND LOW POI
4 POINT CLOUD VOXELIZATION
5 CHALLENGE 4: BUILT COVERAGE

The 3D Python Workflow in the context of LiDAR City Models. We start with the Environment Set up (Step 1) and 3D Data Preparation (Step 2). Once this is done, we move on to Python Automation (Step 3), with a specific part dealing with 3D Python Challenges (Step 4), such as Parcel Surface or Point Of Interest Queries. © Author

To summarize the new skills you unlocked, you can now efficiently address the following challenges:

1. Combining Open-Source Software with Python in a coherent workflow;
2. Gathering Open datasets and preparing them before processing;
3. Processing 3D Data Modalities, especially 3D Point Clouds, 3D Meshes and 3D Voxels with 3D Python;
4. Using critical aspects of each 3D Modality for Advanced Geospatial Analysis while optimizing the code;
5. Extracting Key insights as a City Planner to better understand a local area based on the classified point cloud.

If you feel in control and able to address these different aspects, then you are on the path to becoming a great 3D Geospatial Professional! The only thing to do is to keep up the effort and push existing boundaries.

## 🤿 Going Further

But the learning journey does not end here. Our lifelong search begins, and

But the learning journey does not end here. Our lifelong search begins, and future steps will dive into deepening 3D Voxel work, exploring semantics, CityGML, CityJSON, and especially how to get from Clever to Smart Cities. On top, we will analyze point clouds with deep learning techniques and unlock advanced 3D LiDAR analytical workflows. A lot to be excited about!

#point-cloud #3d-modeling #python #gis #deep-dives